

Supplementary Materials: All Emulators are Wrong, Many are Useful, and Some are More Useful Than Others

A Reproducible Comparison of Computer Model Surrogates

Kellin N. Rumsey

Devin Francom

Graham C. Gibson

Reid Morris

Table of contents

1	Section SM1: R Scripts	2
2	Section SM2: Summary of Emulator Performance	2
2.1	Overall Performance Summary	2
2.2	Best Emulators for Each Function	5
2.2.1	Test Functions	5
2.2.2	Real Datasets	7
3	Section SM3: Reproducible Simulation Studies	9
3.1	Seed Transformations	9
3.2	The Fallback Model	9
3.3	Simulation Study Output	10
3.4	Datasets	10
3.5	Test Functions (and Characteristics)	12
4	Section SM4: The Rashomon Effect and Ensembling	14
5	Section SM5: Additional Figures and Analysis	15
5.1	Fraction of Variance Unexplained	15
5.1.1	FVU Rankplots	16
5.1.2	FVU Performance Profiles	20
5.1.3	FVU Heatmaps	25
5.1.4	FVU Boxplots	32
5.2	Continuous Ranked Probability Scores	33
5.2.1	CRPS Rankplots (data only)	33
5.2.2	CRPS Performance Profiles (testfuncs only)	35
5.3	Computation Time	38
5.3.1	Time Performance Profiles	38
5.3.2	Time Paretoplots	43
5.4	Other Metrics	44
	References	46

1 Section SM1: R Scripts

Most of the figures in the main paper can be recreated by running the script https://github.com/knrumsey/duqling_results/R/duqling/duqling_paper_main.R. In addition, the exact implementation for each of the 32 emulation methods can be found in the `emulators.R` file at https://github.com/knrumsey/duqling_results/R/duqling/emulators.R. Alternatively the implementation functions can be found using the `duqling` package.

```
# Install and load duqling
# devtools::install_github("knrumsey/duqling")
library(duqling)
funcs <- get_emulator_functions("bass")
fit_bass <- funcs$fit_func
pred_bass <- funcs$pred_func
```

2 Section SM2: Summary of Emulator Performance

The package makes it easy to conduct additional analyses. We start by loading the data into the global environment. Then we use the `process_sim_study()` function to get the data frames ready for the `duqling` functions, which expect an object of class `duq_sim_study`.

```
# Sim study on the 60 test functions
data("sim_study_testfuncs")
duq_testfuncs <- process_sim_study(sim_study_testfuncs)

# Sim study on the 40 test functions
data("sim_study_realdata")
duq_realdata <- process_sim_study(sim_study_realdata)
```

2.1 Overall Performance Summary

For each of the 32 surrogate modeling methods, we look at

- Average runtime
- CRPS: winrate, top10rate, and average soft-relative scores
- FVU: winrate, top10rate, and average soft-relative scores

The average soft-relative scores give us a rough measure of practical performance. For emulator i and simulation scenario j we define

$$t_{i,j} = \frac{\text{metric}_{i,j} + \epsilon}{\min_i \{\text{metric}_{i,j}\} + \epsilon},$$

and the average soft-relative score is simply

$$\text{Soft-Rel-Score}_i = J^{-1} \sum_{j=1}^J \min(t_{i,j}, U)$$

where ϵ represents a point at which the metric should be considered practically 0 and U is a truncation point for robustness.

These metrics can be summarized in `duqling` with the code.

```
duq_total <- join_sim_study(duq_testfuncs, duq_realdata)
table <- summarize_sim_study(duq_total,
  summarize="time", summary_fun = function(val) c(mean=mean(val)),
  win_rate = c("CRPS", "FVU"),
  topK_rate = c("CRPS", "FVU"), K=10,
  soft_rel = c("CRPS", "FVU"),
  epsilon=c(0.001, 0.001), upper_bound=c(1e4, 1e4))
```

For readability, we have broken this into two tables for CRPS and FVU separately, given below.

Method	Time (mean)	CRPS (win rate)	CRPS (top 10%)	CRPS (soft relative)
alcgp	9.801	0.000e+00	0.018	91.249
apce	72.261	0.119	0.372	20.663
bart	21.448	0.040	0.361	18.459
baseline	0.079	0.026	0.032	165.767
bass	22.334	0.034	0.625	6.549
bcart	28.001	0.000e+00	0.015	115.181
bcmgp	148.480	0.002	0.384	27.454
blasso	0.203	0.000e+00	0.006	179.515
blm	0.104	0.000e+00	0.021	108.339
bnn	4056.830	0.003	0.074	464.951
bootrf	458.141	0.000e+00	0.141	45.539
bppr	28.401	0.065	0.735	10.628
btreelm	32.645	5.882e-04	0.131	77.220
confrf	9.596	0.010	0.215	39.273
deepgp	4128.560	0.065	0.421	26.443
fitcgp	18.881	0.053	0.224	377.525
gbc	57.086	0.103	0.448	23.698
gp	20.971	0.020	0.627	20.311
gpytorch	29.452	0.020	0.482	331.479
hetgp	211.466	0.038	0.639	20.350
lagp	72.021	0.002	0.244	73.769
mpgp	1106.348	0.000e+00	0.205	43.118
ngboost	155.670	0.001	0.086	95.760
qbass	356.533	0.005	0.544	10.504
rffgp	5.807	0.030	0.179	109.502
rgasp	360.097	0.218	0.781	18.279
rvm	400.524	0.006	0.237	74.625
spce	1096.072	0.023	0.162	432.840
svecgp	85.311	0.126	0.677	20.037
svigp	45.917	0.035	0.572	15.640
tbass	423.510	0.022	0.640	8.650
treegp	1289.831	0.041	0.622	57.468

```

# ---- FVU TABLE ----
table_fvu <- summarize_sim_study(
  duq_total,
  summarize = "time", summary_fun = function(val) c(mean = mean(val)),
  win_rate = "FVU",
  topK_rate = "FVU", K = 10,
  soft_rel = "FVU", epsilon = 0.001, upper_bound=1e4
)

# Convert to numeric
for(j in 2:ncol(table_fvu)){
  table_fvu[,j] <- as.numeric(table_fvu[,j])
}

# Format numeric columns
table_fvu$time_mean <- fmt(table_fvu$time_mean)
table_fvu$FVU_winrate <- fmt(table_fvu$FVU_winrate)
table_fvu$FVU_top10rate <- fmt(table_fvu$FVU_top10rate)
table_fvu$FVU_softrel <- fmt(table_fvu$FVU_softrel)

# Rename columns
colnames(table_fvu) <- c(
  "Method",
  "Time (mean)",
  "FVU (win rate)",
  "FVU (top 10%)",
  "FVU (soft relative)"
)

knitr::kable(table_fvu, align = "lcccc")

```

Method	Time (mean)	FVU (win rate)	FVU (top 10%)	FVU (soft relative)
alcgp	9.801	0.000e+00	0.026	120.738
apce	72.261	0.117	0.382	23.072
bart	21.448	0.031	0.279	10.384
baseline	0.079	0.026	0.048	437.140
bass	22.334	0.043	0.638	8.190
bcart	28.001	0.000e+00	0.021	107.367
bcmgp	148.480	0.003	0.326	36.306
blasso	0.203	0.000e+00	0.005	275.108
blm	0.104	0.000e+00	0.024	195.579
bnn	4056.830	0.001	0.042	596.633
bootrf	458.141	8.824e-04	0.157	43.981
bppr	28.401	0.038	0.744	7.472
btreelm	32.645	2.941e-04	0.097	35.503
confrf	9.596	0.010	0.196	35.834
deepgp	4128.560	0.068	0.431	22.464

Method	Time (mean)	FVU (win rate)	FVU (top 10%)	FVU (soft relative)
fitcgp	18.881	0.019	0.238	604.384
gbc	57.086	0.064	0.434	14.080
gp	20.971	0.043	0.823	8.574
gpytorch	29.452	0.011	0.452	645.535
hetgp	211.466	0.034	0.722	8.853
lagp	72.021	0.004	0.216	74.608
mpgp	1106.348	5.882e-04	0.146	153.458
ngboost	155.670	0.002	0.066	140.235
qbass	356.533	0.006	0.441	20.573
rffgp	5.807	0.025	0.171	532.552
rgasp	360.097	0.337	0.921	2.115
rvn	400.524	0.010	0.248	29.957
spce	1096.072	0.023	0.190	461.980
svecgp	85.311	0.141	0.763	22.105
svigp	45.917	0.008	0.542	20.621
tbass	423.510	0.019	0.542	15.218
treegp	1289.831	0.083	0.777	24.137

The reader may be interested in recreating these tables after filtering to a specific set of simulation scenarios, but we omit this here for brevity.

2.2 Best Emulators for Each Function

To summarize where each emulator performs particularly well, we identify methods that attain the single best CRPS in at least one case. We encourage the reader to filter the simulation study objects before running this code, which we don't do here for the sake of brevity.

2.2.1 Test Functions

```
# Ensure CRPS ranks exist
duq_testfuncs_ranked <- rank_sim_study(duq_testfuncs, metric = "CRPS")

df <- duq_testfuncs_ranked$df

# Keep only winners
best_df <- df[df$CRPS_rank == 1, , drop = FALSE]

# Keep only scenarios with a unique winner
scenario_counts <- ave(best_df$method, best_df$sim_scenario, FUN = length)
best_df <- best_df[scenario_counts == 1, , drop = FALSE]

# Build table: one row per function
ids <- sort(unique(best_df$id))

methods_best <- sapply(ids, function(z) {
```

```

paste(sort(unique(best_df$method[best_df$id == z])), collapse = ", ")
})

n_methods_best <- sapply(ids, function(z) {
  length(unique(best_df$method[best_df$id == z]))
})

total_wins <- sapply(ids, function(z) {
  sum(best_df$id == z)
})

best_table_testfuncs <- data.frame(
  id = ids,
  n_methods_best = n_methods_best,
  methods_best = methods_best,
  stringsAsFactors = FALSE
)

rownames(best_table_testfuncs) <- NULL
colnames(best_table_testfuncs) <- c("Function", "# of Best", "Best Emulators")

knitr::kable(best_table_testfuncs, align = c("l", "c", "l"))

```

Function	# of Best	Best Emulators
banana	1	apce
borehole	7	apce, bass, bppr, fitcgp, rgasp, svecgp, svigp
borehole_low_fidelity	7	apce, bass, bppr, fitcgp, rgasp, svecgp, tbass
cantilever_D	4	bppr, deepgp, gbc, svigp
cantilever_S	3	deepgp, gbc, svigp
circuit	8	apce, bass, bppr, fitcgp, rffgp, rgasp, svecgp, tbass
const_fn	3	baseline, bass, rffgp
const_fn15	3	baseline, bass, rffgp
const_fn3	3	baseline, bass, rffgp
crater	4	bnn, bppr, gbc, svigp
cube3	4	bart, confrf, gbc, treegp
cube3_rotate	2	deepgp, gbc
cube5	3	bart, gbc, treegp
detpep_curve	5	apce, bppr, fitcgp, rffgp, rgasp
detpep8	7	bass, bppr, fitcgp, rgasp, svecgp, svigp, tbass
dms_additive	4	apce, bppr, rgasp, svecgp
dms_complicated	8	apce, bcmgp, fitcgp, gp, gpytorch, hetgp, svecgp, svigp
dms_harmonic	1	apce
dms_radial	1	apce
dms_simple	2	apce, fitcgp
ebola	5	bppr, rgasp, svecgp, svigp, tbass
forrester1	5	apce, bppr, fitcgp, rffgp, rgasp

Function	# of Best	Best Emulators
forrester1_low_fidelity	5	apce, bppr, fitcgp, rffgp, rgasp
foursquare	7	bart, bass, confrf, deepgp, qbass, tbass, treegp
friedman	5	apce, bass, bppr, rgasp, svecgp
friedman10	5	apce, bass, bppr, svecgp, tbass
friedman20	4	bass, bppr, rgasp, svecgp
gamma_mix	8	bass, bnn, bppr, gbc, hetgp, qbass, tbass, treegp
Gfunction	5	bass, gpytorch, rgasp, svigp, tbass
Gfunction12	7	bass, bppr, gbc, rgasp, spce, svigp, tbass
Gfunction18	7	bcmgp, deepgp, gbc, qbass, rgasp, spce, tbass
Gfunction6	5	bass, bppr, gbc, rgasp, svigp
grlee1	9	bart, bcmgp, fitcgp, gpytorch, lagp, qbass, rgasp, svigp, tbass
grlee2	7	bass, fitcgp, gp, gpytorch, hetgp, svecgp, svigp
grlee6	5	bass, qbass, rgasp, svecgp, tbass
ignition	2	deepgp, gbc
ishigami	5	apce, gp, hetgp, rgasp, svecgp
lim_non_polynomial	4	bppr, fitcgp, rgasp, svecgp
lim_polynomial	3	apce, fitcgp, rffgp
multivalley	4	bart, bass, rgasp, tbass
onehundred	5	gp, hetgp, rgasp, rvm, spce
oo15	2	rgasp, spce
park4	8	apce, bass, bppr, fitcgp, gp, hetgp, rgasp, svecgp
park4_low_fidelity	8	bass, fitcgp, gp, hetgp, rffgp, rgasp, svecgp, tbass
permdb	5	confrf, deepgp, fitcgp, ngboost, treegp
piston	5	bppr, rgasp, svecgp, svigp, tbass
pollutant_uni	5	apce, bass, bppr, rgasp, tbass
rabbits	8	bass, bppr, fitcgp, gp, gpytorch, hetgp, svecgp, svigp
ripples	8	bcmgp, deepgp, gp, gpytorch, hetgp, rgasp, svecgp, treegp
robot	5	gp, hetgp, rgasp, svecgp, treegp
sharkfin	2	bart, tbass
short_column	6	bppr, fitcgp, rgasp, svecgp, svigp, treegp
simple_poly	3	apce, fitcgp, rffgp
squiggle	5	deepgp, gbc, gpytorch, svecgp, treegp
star2	5	confrf, deepgp, gbc, rgasp, svecgp
steel_column	7	bass, bppr, fitcgp, gpytorch, rgasp, svecgp, svigp
sulfur	6	deepgp, gbc, gp, gpytorch, hetgp, svigp
twin_galaxies	5	deepgp, fitcgp, gpytorch, hetgp, svecgp
welch20	4	apce, bppr, qbass, rgasp
wingweight	6	bppr, fitcgp, gpytorch, rgasp, svigp, tbass

2.2.2 Real Datasets

Though not shown, the same code is used as above, with `duq_realdata` in place of `duq_testfuncs`.

Function	# of Best	Best Emulators
acme_climate	3	bart, gbc, svecgp
diablo_canyon_plume	2	gbc, svecgp
diffusion_1D	1	rgasp
diffusion_2D	1	gbc
discoflux_flyer	1	gbc
e3sm_mcar	1	svecgp
e3sm_mnar	1	svecgp
fair_climate_ssp1-2.6_year2200	2	hetgp, rgasp
fair_climate_ssp2-4.5_year2200	2	rgasp, svecgp
fair_climate_ssp3-7.0_year2200	1	rgasp
flyer_plate104	5	bart, gbc, qbass, rgasp, tbass
flyerPTW1	5	bppr, gbc, hetgp, rgasp, svecgp
flyerPTW2	1	rgasp
icf1	1	svecgp
icf2	1	svecgp
jc	1	rgasp
nuclear_data	1	bppr
okc_plume	1	gbc
pbx9501_gold	6	deepgp, gbc, gp, rgasp, svecgp, treegp
pbx9501_nickel	2	gpytorch, rgasp
pbx9501_ss304	3	gpytorch, rgasp, svecgp
pbx9501_uranium	4	gbc, hetgp, svecgp, treegp
plate_deformation	7	bppr, btrelm, deepgp, gbc, rgasp, svecgp, treegp
ptw1	4	gbc, rgasp, svecgp, treegp
ptw2	1	gbc
SLOSH_high	2	gbc, svecgp
SLOSH_low	1	gbc
SLOSH_mid	1	gbc
spectral1	2	gp, hetgp
spectra2	3	gp, hetgp, treegp
stochastic_sir	1	gbc
strontium_plume_p104	1	svecgp
strontium_plume_p4b	7	btrelm, confrf, gbc, hetgp, ngboost, qbass, svecgp
taylor_cylinder1	3	bass, gbc, rgasp
taylor_foot	2	gbc, rgasp
taylor_length	1	rgasp
wind_speed	6	bnn, gp, gpytorch, qbass, rgasp, svecgp
Z_machine_max_vel_all	1	svecgp
Z_machine_max_vel1	1	svecgp
Z_machine_max_vel2	1	svecgp

3 Section SM3: Reproducible Simulation Studies

3.1 Seed Transformations

One of the primary design principles of `duqling::run_sim_study()` is reproducibility. For any combination of (i) test function, (ii) n_{train} , (iii) design type, (iv) noise to signal ratio, and (v) replication number, the generated training data (inputs and response) are identical. This is true, even when using slightly different function calls. For example,

```
results1 <- run_sim_study(fit_func, pred_func,
                        fnames=c("borehole", "ishigami", "banana"),
                        NSR=c(0, 0.1),
                        design_type="LHS",
                        replications=10)

results2 <- run_sim_study(fit_func, pred_func,
                        fnames=c("ishigami", "borehole"),
                        NSR=0,
                        design_type="LHS",
                        replications=c(1, 7, 15))
```

the rows of `results1` and `results2` corresponding to the same simulation scenario will have identical results (with minor exceptions such as timing data) even though they were run in a different order internally.

To accomplish this, we use a rolling polynomial hash (also called Rabin fingerprinting) to generate a unique seed for each simulation scenario,

$$\text{seed} = \left(\sum_{j=0}^6 B^j s_j \right) \bmod M \quad (1)$$

for prime numbers $B = 101$ and $M = 100030001$, where the set $\{s_0, \dots, s_j\}$ describes the simulation scenario. This can be computed safely using Horner's rule ([1]). The s_j values are computed as follows: s_0 is the user-defined seed with default 42 (not usually intended to be modified), $s_1 = n_{\text{train}}$, s_2 is the replication index. To obtain s_3 , we hash each function name to a unique integer (with high probability) again using the Rabin fingerprint (with $B = 31$) after mapping each character to a unique integer. The pair (s_4, s_5) encodes the noise to signal ratio (NSR) with at least four decimal places of precision as follows:

$$s_4 = \begin{cases} 99999, & NSR = 0 \\ \lceil -10^4 \times \log(NSR) \rceil, & NSR < 1 \\ 0, & NSR \geq 1 \end{cases}, \quad s_5 = \begin{cases} 0, & NSR < 1 \\ \lceil 10^4 \times NSR \rceil, & NSR \geq 1 \end{cases}. \quad (2)$$

Finally, $s_6 \in \{1, 2, 3\}$ denotes the design type: LHS, Grid, and Random, respectively.

3.2 The Fallback Model

Even the most robust emulator implementations can occasionally fail, particularly when faced with such a diverse and challenging set of test functions and datasets. To ensure that simulation studies proceed smoothly and results remain well-defined, `duqling::run_sim_study()` incorporates

a fallback strategy. If one of the emulator methods fails during fitting or prediction, it is temporarily replaced with a baseline “null model” that predicts by fitting a Gaussian distribution to the marginal of the response variable from the training data (ignoring the inputs). Specifically, the predictive distribution for each test point is a Student’s t distribution with $n - 1$ degrees of freedom, mean $\bar{y}$, and variance $S_y^2(1 + 1/n)(n - 1)/(n - 3)$, where n , $\bar{y}$, and S_y^2 are the training set size, mean, and sample variance, respectively. When a failure occurs, it is logged in the `failure_type` column of the returned data-frame one of the following values: “none”, “fit”, or “pred”.

3.3 Simulation Study Output

The `run_sim_study()` function returns an R data frame in which each row corresponds to the outcome of a single simulation scenario, replication, and emulator method. The returned columns include:

- **method**: Short name of the fitted emulator; if no name is supplied, a unique label (`method<indx>`) is generated automatically.
- **fname**: The test function name (from `duqling::quack()`).
- **input_dim**: Input dimension of the test function.
- **n**: Number of training points used.
- **NSR**: Noise-to-signal ratio applied in data generation.
- **design_type**: Experimental design type ("LHS", "grid", or "random").
- **rep**: Replication index.
- **t_fit**, **t_pred**, **t_tot**: Elapsed wall-clock time (seconds) for fitting, prediction, and total runtime, respectively.
- **failure_type**: Categorical indicator for whether the method failed at fitting, prediction, or "none" if successful.
- **RMSE**: Root mean squared error on the test set.
- **FVU**: Fraction of variance unexplained, computed as RMSE^2 divided by the total variance.
- **COVER**, **MIS**: Coverage rates and mean interval scores for each specified confidence level.
- **IAE_alpha**: Integrated absolute error of empirical coverage across confidence levels, summarizing calibration of predictive intervals.
- **CRPS**: Mean continuous ranked probability score on the test set, along with summary statistics (`CRPS_min`, `CRPS_Q1`, `CRPS_med`, `CRPS_Q3`, `CRPS_max`) describing the distribution of CRPS across test points.

Additional columns may be present depending on the metrics and simulation options selected by the user. The full results for the simulation studies shown in Section 4 are included in the associated GitHub repository.

3.4 Datasets

A full list of the real datasets used in this comparison is given in the table below. We are hopeful that all of these will be available in `duqling` soon. You can check which emulation datasets are available in `duqling` via the command:

```
data_quack(raw=FALSE, response_type="uni")
```

	dname	input_dim	input_cat_dim	n	response_type
1	e3sm	2	0	48602	uni
2	e3sm_mcar	2	0	10000	uni

3	e3sm_mnar	2	0	9122	uni
4	stochastic_sir	4	0	2000	uni
5	SLOSH_low	5	0	4000	uni
6	SLOSH_mid	5	0	4000	uni
7	SLOSH_high	5	0	4000	uni
8	pbx9501_gold	6	0	500	uni
9	pbx9501_ss304	6	0	500	uni
10	pbx9501_nickel	6	0	500	uni
11	pbx9501_uranium	6	0	500	uni
12	Z_machine_max_vel1	6	0	5000	uni
13	Z_machine_max_vel2	6	0	5000	uni
14	flyer_plate104	11	0	1000	uni
15	strontium_plume_p4b	20	0	300	uni
16	strontium_plume_p104	20	0	300	uni
17	Z_machine_max_vel_all	30	0	5000	uni
18	fair_climate_ssp1-2.6_year2200	45	0	1001	uni
19	fair_climate_ssp2-4.5_year2200	45	0	1001	uni
20	fair_climate_ssp3-7.0_year2200	45	0	1001	uni
21	fair_climate_ssp1-2.6	46	0	56056	uni
22	fair_climate_ssp2-4.5	46	0	56056	uni
23	fair_climate_ssp3-7.0	46	0	56056	uni

A particular dataset can be read into R (internet connection is required) with the command:

```
emu_data <- get_emulation_data("pbx9501_gold")
X <- emu_data$X
y <- emu_data$y
```

The full list of datasets is given below.

Dataset	n	p	In duqling	Reference
plate_deformation	138	7	No	[2]
wind_speed	200	17	No	
strontium_plume_p4b	300	20	Yes	[3]
strontium_plume_p104	300	20	Yes	[3]
spectral1	500	4	No	[4]
spectra2	500	4	No	[4]
pbx9501_gold	500	6	Yes	[5]
pbx9501_ss304	500	6	Yes	[5]
pbx9501_nickel	500	6	Yes	[5]
pbx9501_uranium	500	6	Yes	[5]
ptw1	500	10	No	[6]
discoflux_flyer	914	6	No	[7]
taylor_foot	962	11	No	
taylor_length	962	11	No	
jc	1000	5	No	[8]
flyerPTW1	1000	11	No	
flyerPTW2	1000	11	No	

Dataset	n	p	In duqling	Reference
flyer_plate104	1000	11	Yes	
diffusion_1D	1000	62	No	[9]
fair_climate_ssp1-2.6_year2200	1001	45	Yes	[10]
fair_climate_ssp2-4.5_year2200	1001	45	Yes	[10]
fair_climate_ssp3-7.0_year2200	1001	45	Yes	[10]
taylor_cylinder1	1438	8	No	
stochastic_sir	2000	4	Yes	[11]
diffusion_2D	2000	53	No	[9]
acme_climate	2980	66	No	
ptw2	3701	13	No	[6]
SLOSH_low	4000	5	Yes	[12]
SLOSH_mid	4000	5	Yes	[12]
SLOSH_high	4000	5	Yes	[12]
Z_machine_max_vel1	5000	6	Yes	[13]
Z_machine_max_vel2	5000	6	Yes	[13]
Z_machine_max_vel_all	5000	30	Yes	[13]
e3sm_mnar	9122	2	Yes	[14]
nuclear_data	9307	178	No	[15]
e3sm_mcar	10000	2	Yes	[14]
icf1	10000	5	No	
icf2	10000	5	No	
diablo_canyon_plume	18000	13	No	[16]
okc_plume	22229	5	No	[17]

3.5 Test Functions (and Characteristics)

The `duqling` library of test functions is ever growing, but the list of the 60 test functions studied in this paper can be found as

```
get_paper_funcs()
```

For these 60 functions, we computed several characteristics of interest. Future work could try to link these characteristics (or others not listed here) to emulator performance. Determining, in advance, which surrogate modeling technique is likely to succeed on a given computer model would be of great use to practitioners.

fname	input_dim	Rsquared	MoransI	Sobolev
const_fn	1	0.000	0.020	1.000
forrester1_low_fidelity	1	0.407	0.998	9.785
forrester1	1	0.043	0.982	11.682
grlee1	1	0.604	0.987	16.676
banana	2	0.035	0.955	11.017
dms_additive	2	0.058	0.984	9.895
dms_complicated	2	0.192	0.962	9.831
dms_harmonic	2	0.030	0.932	13.037
dms_radial	2	0.000	0.983	7.092

fname	input_dim	Rsquared	MoransI	Sobolev
dms_simple	2	0.189	0.948	4.768
foursquare	2	0.471	0.931	24.490
grlee2	2	0.022	0.904	16.179
lim_non_polynomial	2	0.643	0.974	5.532
lim_polynomial	2	0.594	0.958	6.212
multivalley	2	0.091	0.701	72.760
ripples	2	0.003	0.807	40.944
simple_poly	2	0.945	0.980	3.798
squiggle	2	0.015	0.868	25.773
star2	2	0.204	0.803	22.482
twin_galaxies	2	0.442	0.962	9.100
const_fn3	3	0.000	-0.022	1.000
cube3_rotate	3	0.198	0.797	10.383
cube3	3	0.060	0.795	31.274
detpep_curve	3	0.795	0.956	4.611
Gfunction	3	0.000	0.883	1.000
ishigami	3	0.199	0.841	11.257
rabbits	3	0.927	0.954	4.710
sharkfin	3	0.246	0.753	41.910
park4_low_fidelity	4	0.956	0.936	3.658
park4	4	0.955	0.919	3.655
pollutant_uni	4	0.827	0.886	5.231
cube5	5	0.622	0.785	19.961
friedman	5	0.770	0.877	4.856
short_column	5	0.769	0.824	4.730
cantilever_D	6	0.501	0.701	5.993
cantilever_S	6	0.699	0.827	4.495
circuit	6	0.973	0.855	3.869
Gfunction6	6	0.002	0.562	1.000
grlee6	6	0.302	0.433	40.708
crater	7	0.647	0.758	5.090
gamma_mix	7	0.094	0.091	16.440
piston	7	0.914	0.819	4.093
borehole_low_fidelity	8	0.947	0.786	3.766
borehole	8	0.947	0.791	3.729
detpep8	8	0.802	0.752	4.609
ebola	8	0.856	0.748	4.794
robot	8	0.174	0.410	8.614
steel_column	9	0.968	0.762	3.813
sulfur	9	0.549	0.698	4.473
friedman10	10	0.742	0.674	5.064
ignition	10	0.815	0.729	3.937
wingweight	10	0.984	0.758	3.652
Gfunction12	12	0.002	0.275	1.000
const_fn15	15	0.001	0.001	1.000
oo15	15	0.258	0.574	1.000

fname	input_dim	Rsquared	MoransI	Sobolev
permdb	16	0.000	0.101	46.941
Gfunction18	18	0.000	0.139	1.000
friedman20	20	0.753	0.675	4.961
welch20	20	0.598	0.598	6.375
onehundred	100	0.884	0.279	4.361

4 Section SM4: The Rashomon Effect and Ensembling

We have already seen that searching for a single emulator that performs best across all functions is a fruitless endeavor—but perhaps we should not even be focused on finding the best emulator for a single setting. “*Amazing things come from having many good models*” is the sentiment behind the Rashomon effect, an idea that has gained increasing attention in recent years ([18]; [19]; [20]; [21]). The term was originally coined by Leo Breiman to describe the phenomenon in which multiple distinct models explain the data equally well, yet may lead to different conclusions ([22]). While the Rashomon effect is often discussed in the context of noisy or over-parameterized models, we believe it also holds valuable lessons for uncertainty quantification.

Suppose that for a given dataset, we retain all emulators with a CRPS below 0.25 (see Figure below). If we then perform a sensitivity analysis using each of these similarly accurate emulators and they all agree, this serves as a strong sanity check—not just for the conclusions, but also for the models themselves. On the other hand, if the emulators disagree, this immediately signals a need for deeper inspection. For example, imagine that one emulator identifies $\mathbf{x}_1$ as the most important input, $\mathbf{x}_2$ as moderately important, and $\mathbf{x}_3$ as inert—while another emulator ranks $\mathbf{x}_3$ highest, de-emphasizes $\mathbf{x}_1$, and again sees $\mathbf{x}_2$ as moderately important. Such a discrepancy suggests a lack of identifiability between $\mathbf{x}_1$ and $\mathbf{x}_3$, and highlights how choosing a single emulator might obscure meaningful structure in the data. Had we selected only one model, we might have confidently reported that $\mathbf{x}_1$ does not matter—when in fact, the full Rashomon set tells a more nuanced story.

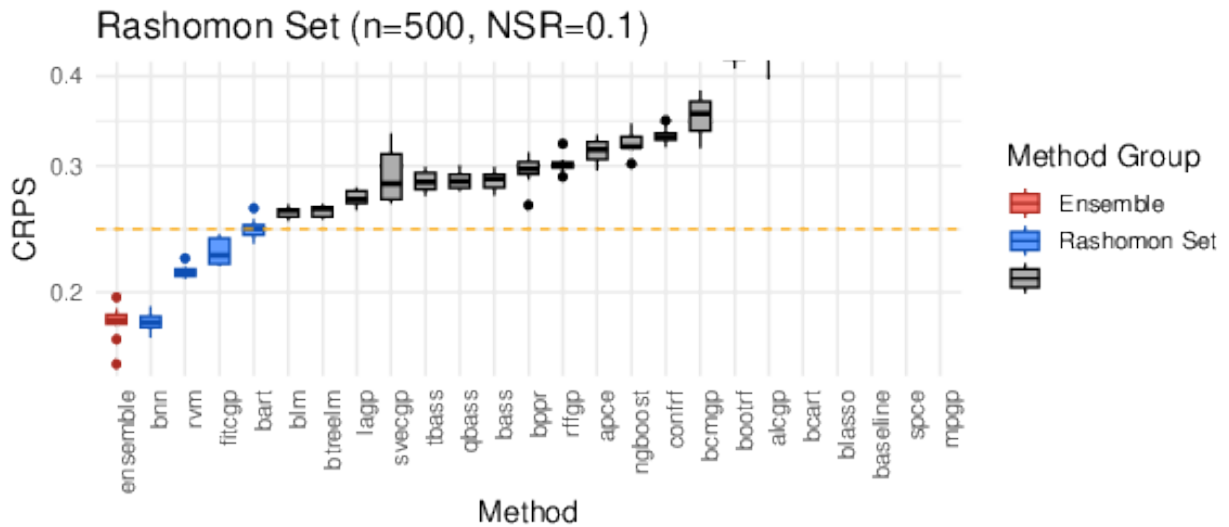


Figure 1: Rashomon plot

5 Section SM5: Additional Figures and Analysis

In this section, we illustrate how the `duqling` framework can be used to explore additional metrics and visualizations beyond those presented in the main text. See Section SM1 and SM2 for details on installing/loading `duqling` and the simulation study objects.

We commonly *filter* the simulation study objects to study specific settings, e.g.,

```
filter_sim_study(duq_testfuncs, n_train=500, NSR=0)
```

The plotting and analysis functions provided by the package include

- `summarize_sim_study()`: Computes summary statistics across simulation scenarios (e.g., mean or median performance), producing a compact view of emulator performance.
- `heatmap_sim_study()`: Visualizes performance metrics (e.g., CRPS, FVU) across scenarios using heatmaps, helping identify patterns across settings.
- `rankplot_sim_study()`: Displays cumulative rank distributions, showing how often each emulator achieves a given rank or better.
- `paretoplot_sim_study()`: Plots trade-offs between metrics (typically accuracy vs. compute time), highlighting Pareto-efficient methods.
- `perfprofile_sim_study()`: Generates performance profile plots, showing the proportion of scenarios for which each method is within a factor of the best.
- `boxplots_sim_study()`: Produces boxplots of performance metrics across scenarios, useful for assessing variability and the Rashomon effect.
- `rank_sim_study()`: Computes rank-based summaries for a specified metric and appends corresponding rank columns.
- `normalize_sim_study()`: Normalizes metrics (e.g., scaling relative to variance or baseline values) to improve comparability across scenarios.
- `relativize_sim_study()`: Computes relative performance metrics by comparing each method to the best-performing method within each scenario.
- `collapse_sim_study()`: Aggregates results across replications or folds, producing a reduced data frame suitable for summary analyses.

5.1 Fraction of Variance Unexplained

In the main paper, we generally analyze CRPS due to our focus on uncertainty quantification. If the focus is on pure predictive performance and not on well-calibrated predictive intervals, then mean squared error (MSE) or root MSE (RMSE) is the usual choice. The fraction of variance unexplained (FVU) is simply a scaled version of MSE which improves interpretability and allows for some level of comparison between simulation scenarios. We calculate FVU as

$$\text{FVU} = \frac{\frac{1}{n_{\text{test}}} \sum_{i=1}^{n_{\text{test}}} (y_{\text{test},i} - \hat{y}_i)^2}{\frac{1}{n_{\text{test}}-1} \sum_{i=1}^{n_{\text{test}}} (y_{\text{test},i} - \bar{y}_{\text{test}})^2}$$

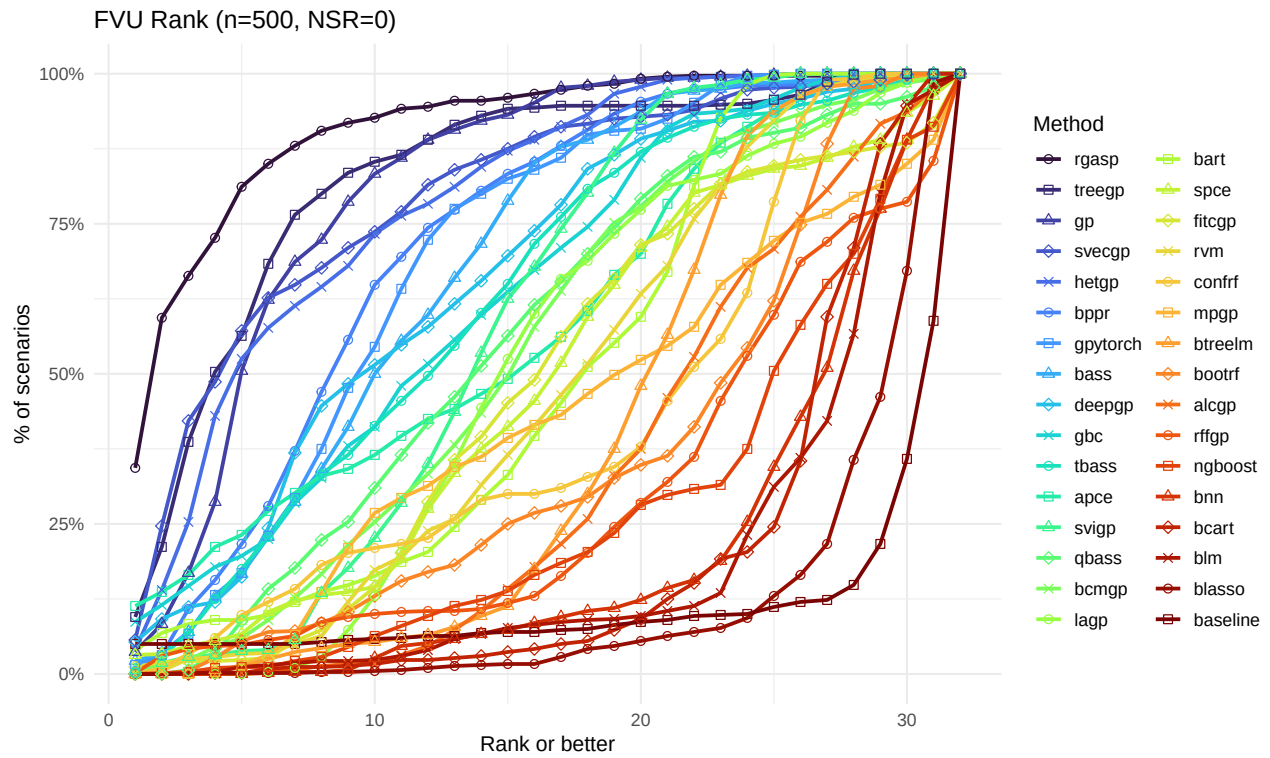
where $\hat{y}_1, \dots, \hat{y}_{n_{\text{test}}}$ are the emulator predictions. This metric (along with RMSE) is automatically calculated during `run_sim_study()` and `run_sim_study_data()`.

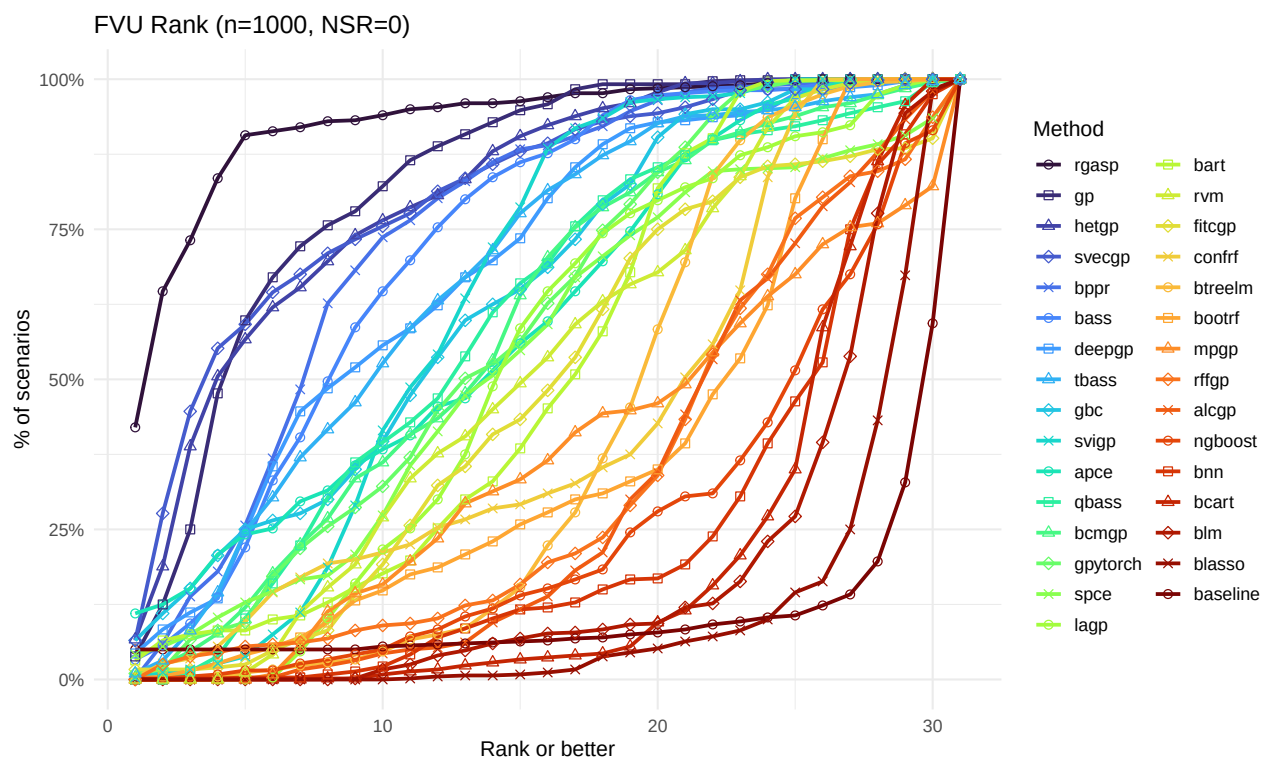
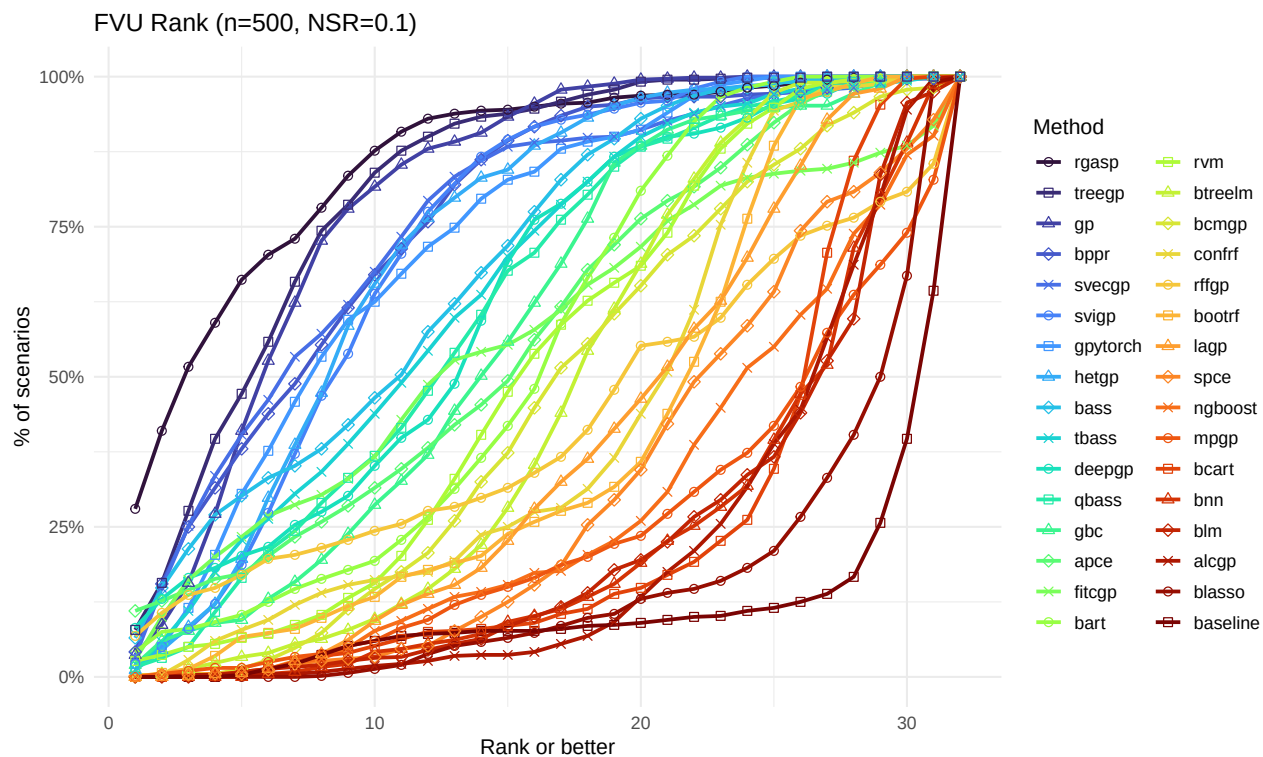
5.1.1 FVU Rankplots

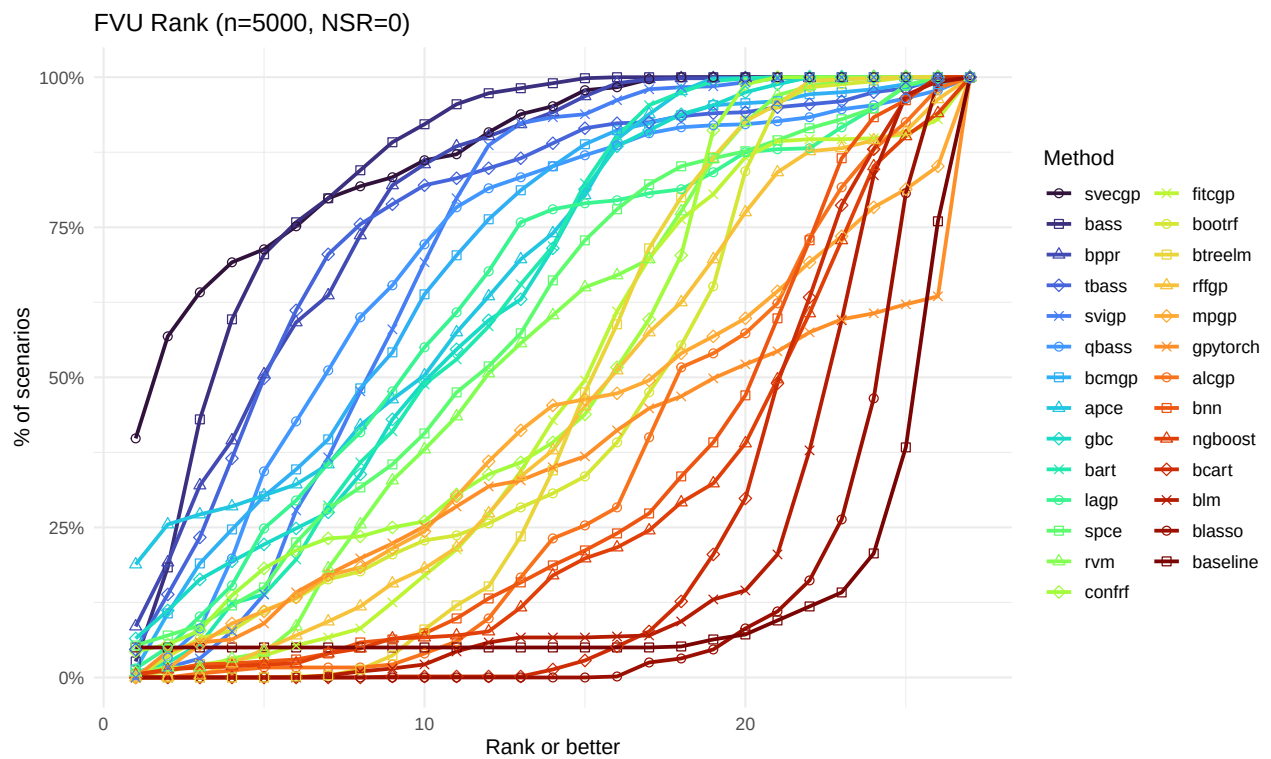
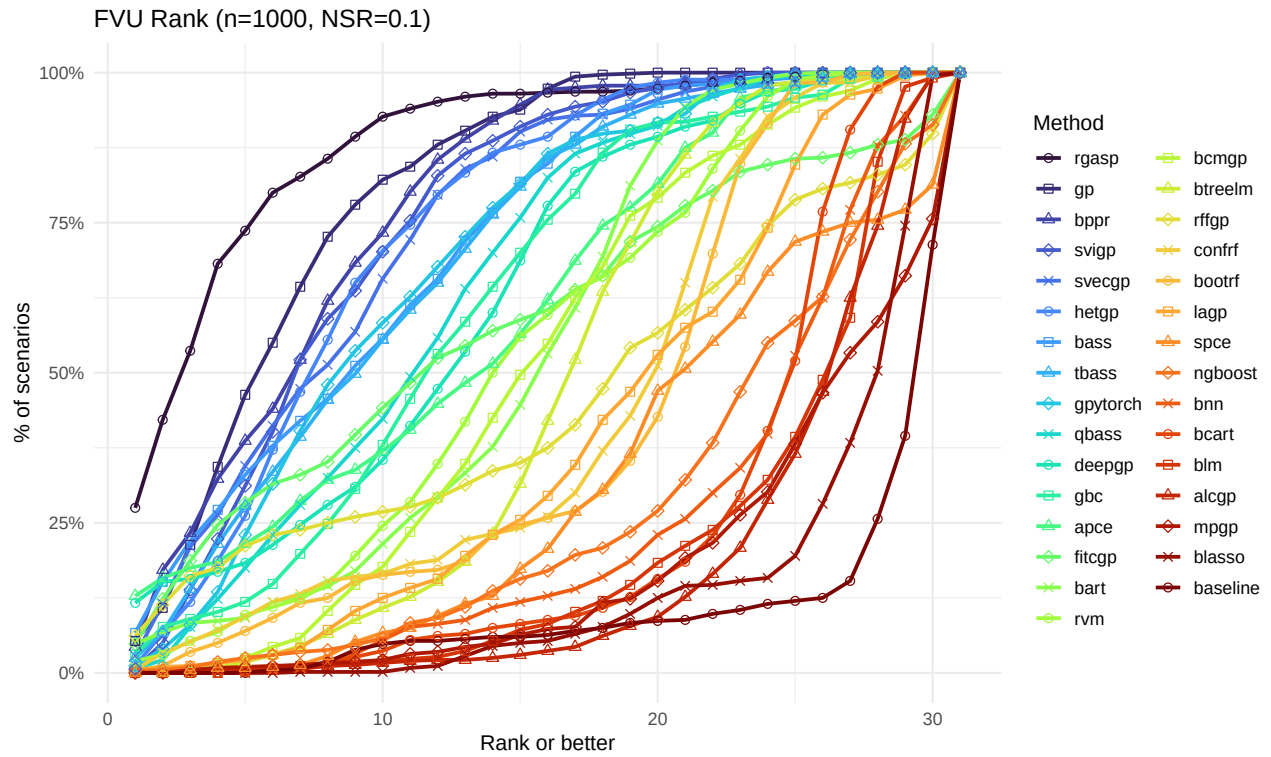
We use the following loop structure to iterate over the five synthetic simulation settings.

```
n_train_vec <- c(500, 500, 1000, 1000, 5000)
NSR_vec      <- c(0, 0.1, 0, 0.1, 0)
for(i in 1:5){
  title <- paste0("FVU Rank (n=", n_train_vec[i], ", NSR=", NSR_vec[i], ")")
  tmp <- filter_sim_study(duq_testfuncs,
                          n_train = n_train_vec[i], NSR = NSR_vec[i])
  p <- rankplot_sim_study(tmp, metric="FVU", title=title)

  # Adjust sizing
  p <- p + ggplot2::theme_minimal(base_size=12)
  plot(p)
}
```

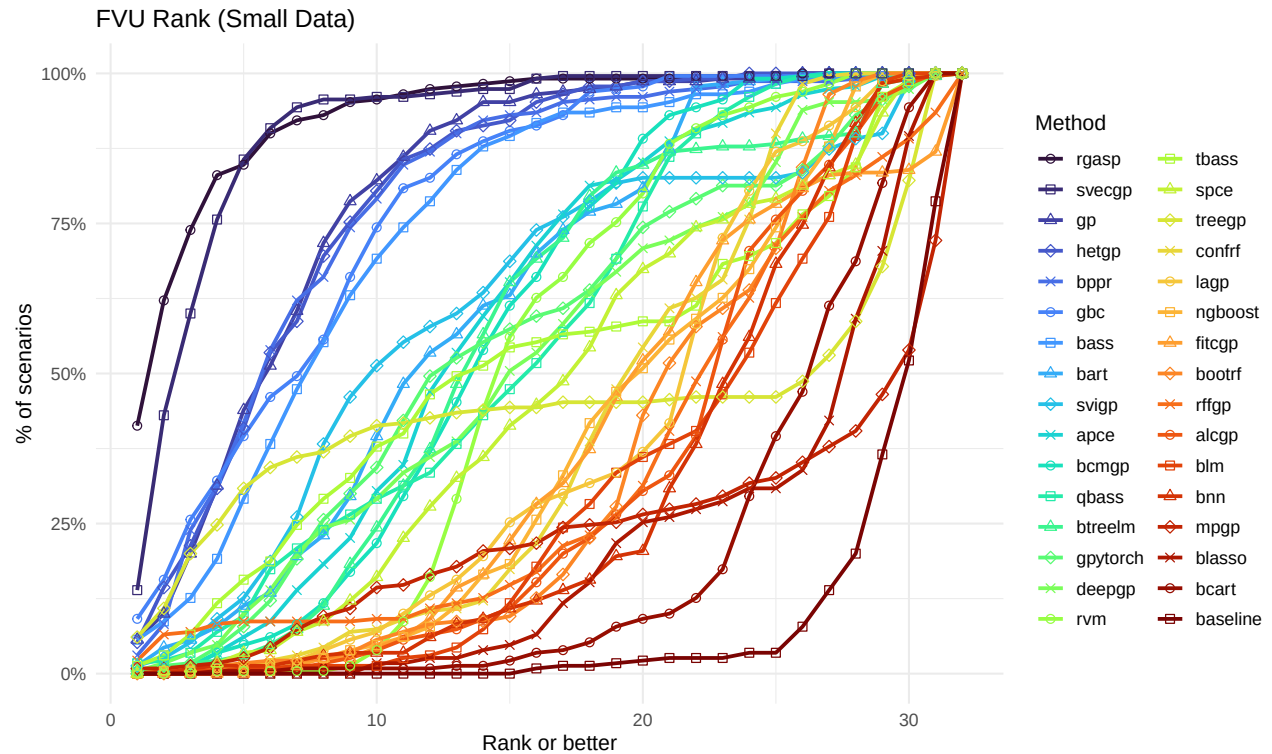




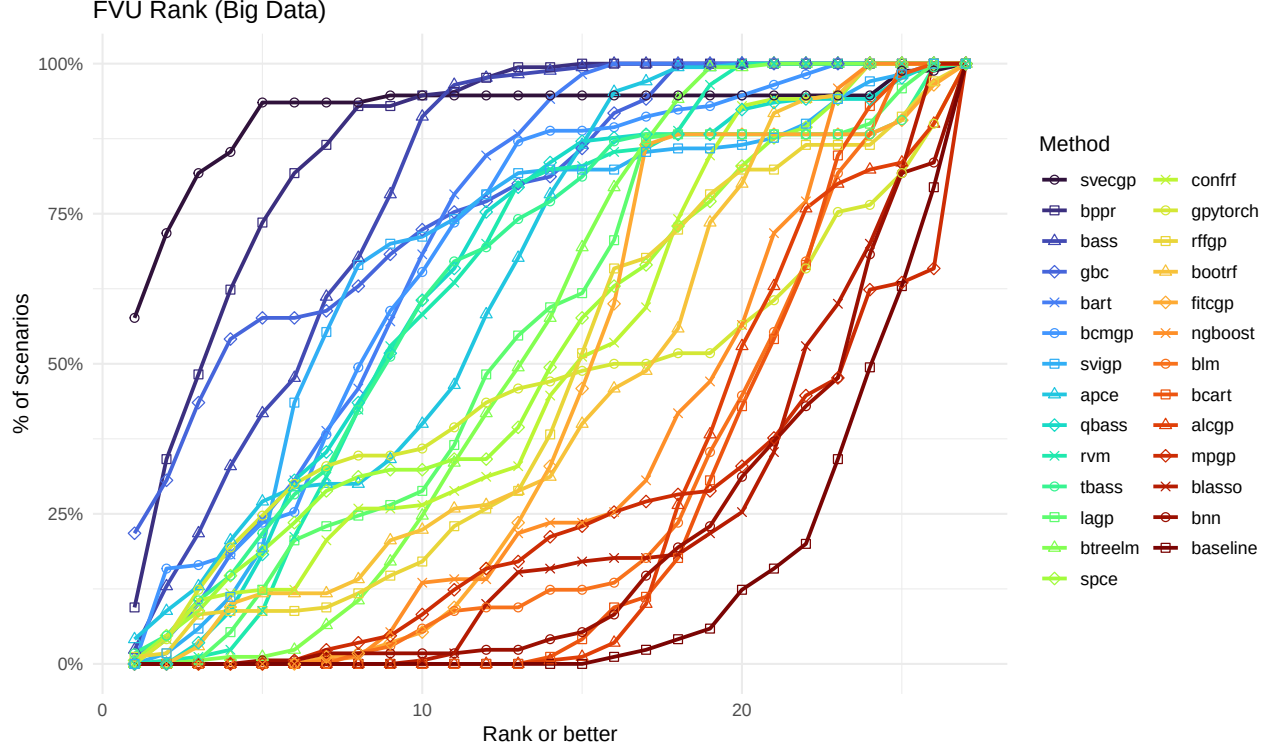


```
# Small data
title <- paste0("FVU Rank (Small Data)")
tmp <- filter_sim_study(duq_realdata,
                        expr = n_train <= 1500)
p <- rankplot_sim_study(tmp, metric="FVU", title=title)
```

```
p <- p + ggplot2::theme_minimal(base_size=12)
plot(p)
```



```
# Large data
title <- paste0("FVU Rank (Big Data)")
tmp <- filter_sim_study(duq_realddata,
                        expr = n_train > 1500)
p <- rankplot_sim_study(tmp, metric="FVU", title=title)
p <- p + ggplot2::theme_minimal(base_size=12)
plot(p)
```



5.1.2 FVU Performance Profiles

Performance profiles come from the literature on benchmarking for optimization research ([23]; [24]). For emulator i and simulation scenario j we define

$$t_{i,j} = \frac{\text{metric}_{i,j} + \epsilon}{\min_i \{\text{metric}_{i,j}\} + \epsilon},$$

and we define

$$\rho_i(\tau) = \frac{1}{J} \sum_{j=1}^J 1(t_{i,j} \leq \tau).$$

When $\epsilon = 0$, the idea is that $\rho_i(\tau)$ tells us the proportion of the time that emulator i is within a factor τ of the best performing emulator across the J different simulation scenarios. We use a small value of $\epsilon = 0.001$ to perform a type of “Laplace smoothing”, which effectively induces the belief that values of the metric (FVU, here) less than this value are all effectively equally good.

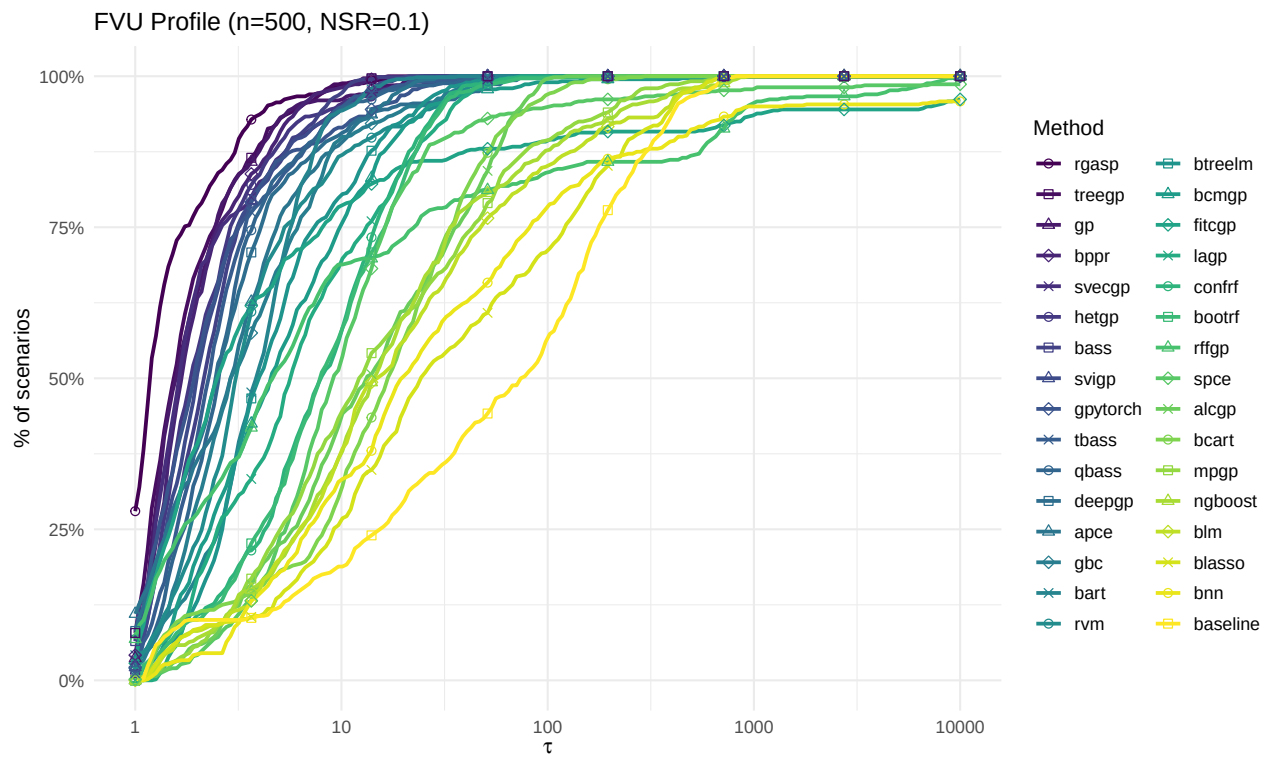
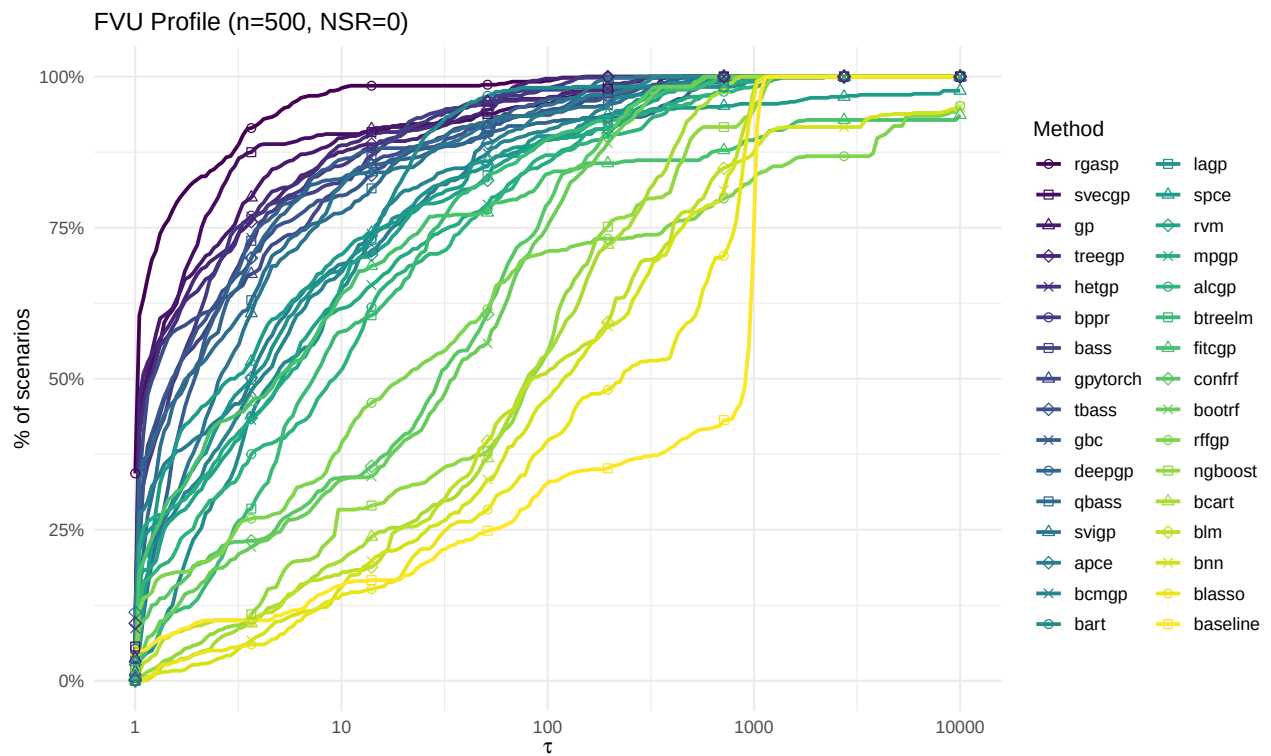
```
n_train_vec <- c(500, 500, 1000, 1000, 5000)
NSR_vec      <- c(0, 0.1, 0, 0.1, 0)
for(i in 1:5){
  title <- paste0("FVU Profile (n=", n_train_vec[i], ", NSR=", NSR_vec[i], ")")
  tmp <- filter_sim_study(duq_testfuncs,
    n_train = n_train_vec[i], NSR = NSR_vec[i])
  p <- perfprofile_sim_study(tmp, metric="FVU",
    relative_epsilon = 0.001, tau_lim=c(1, 10000),
    title=title, color_scale = "viridis")

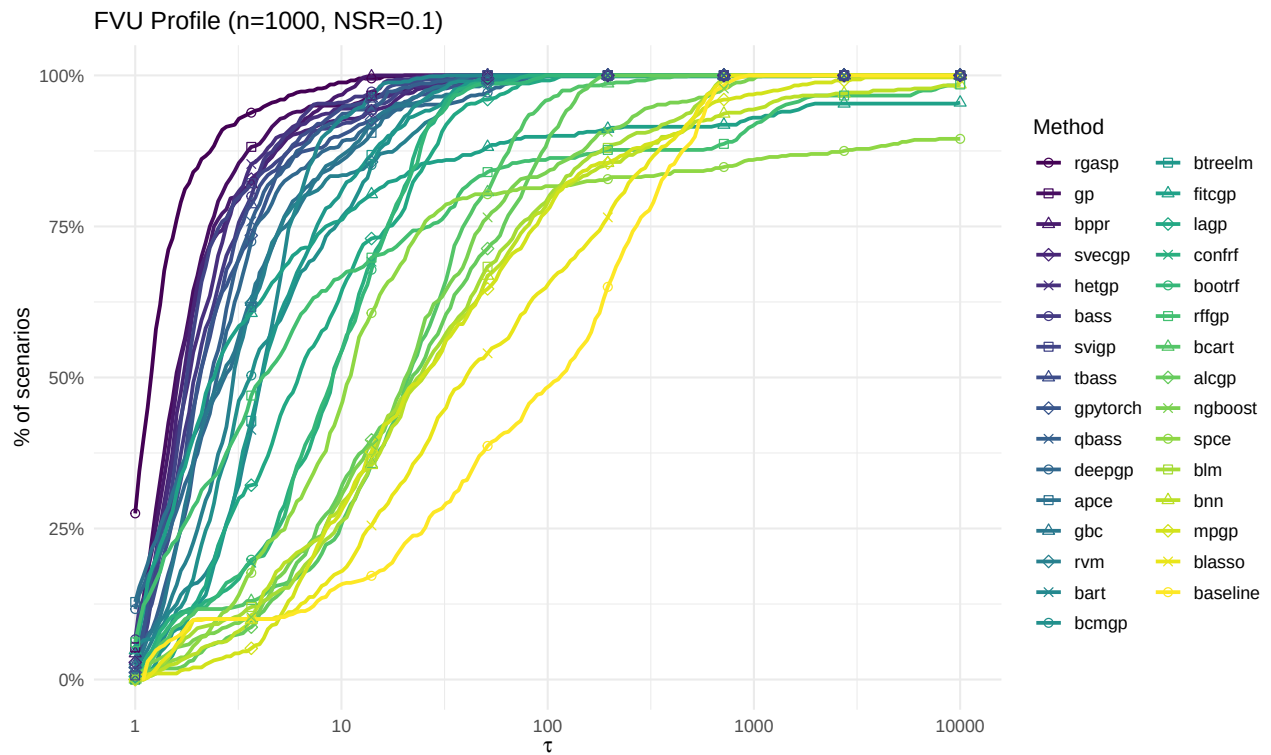
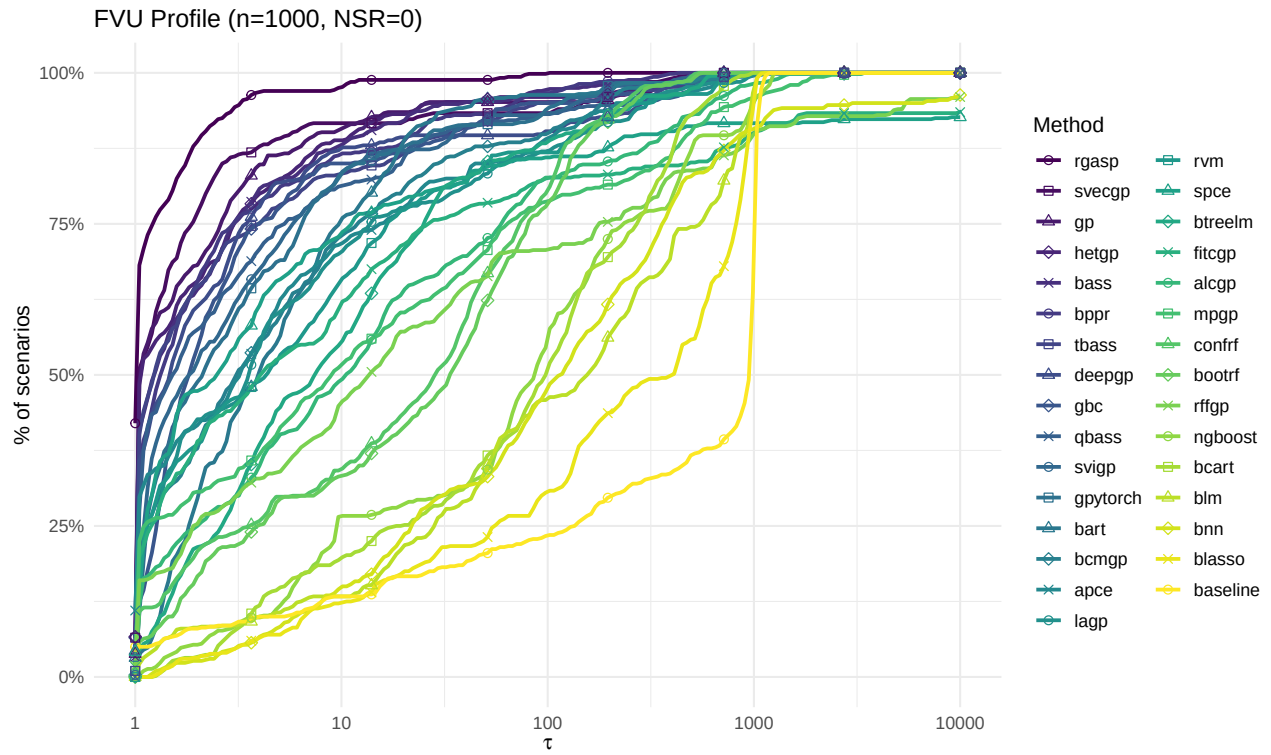
  # Adjust sizing
```

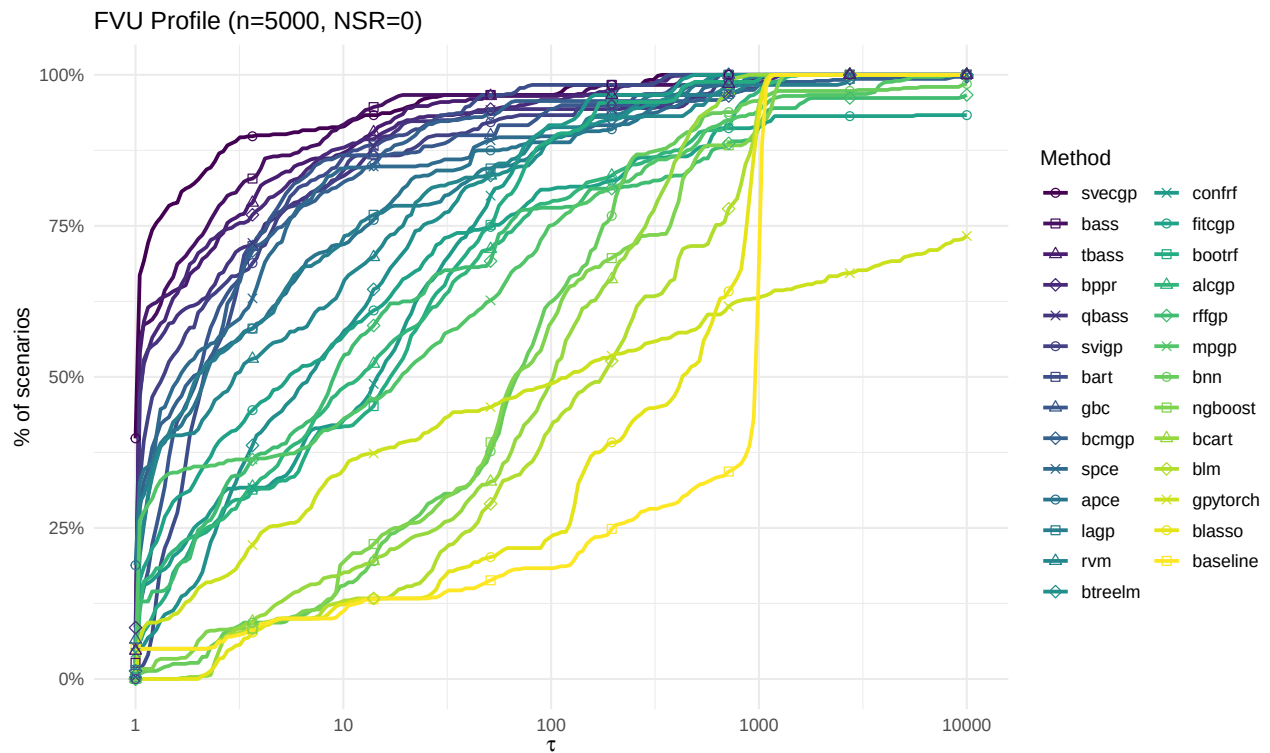
```

p <- p + ggplot2::theme_minimal(base_size=12)
plot(p)
}

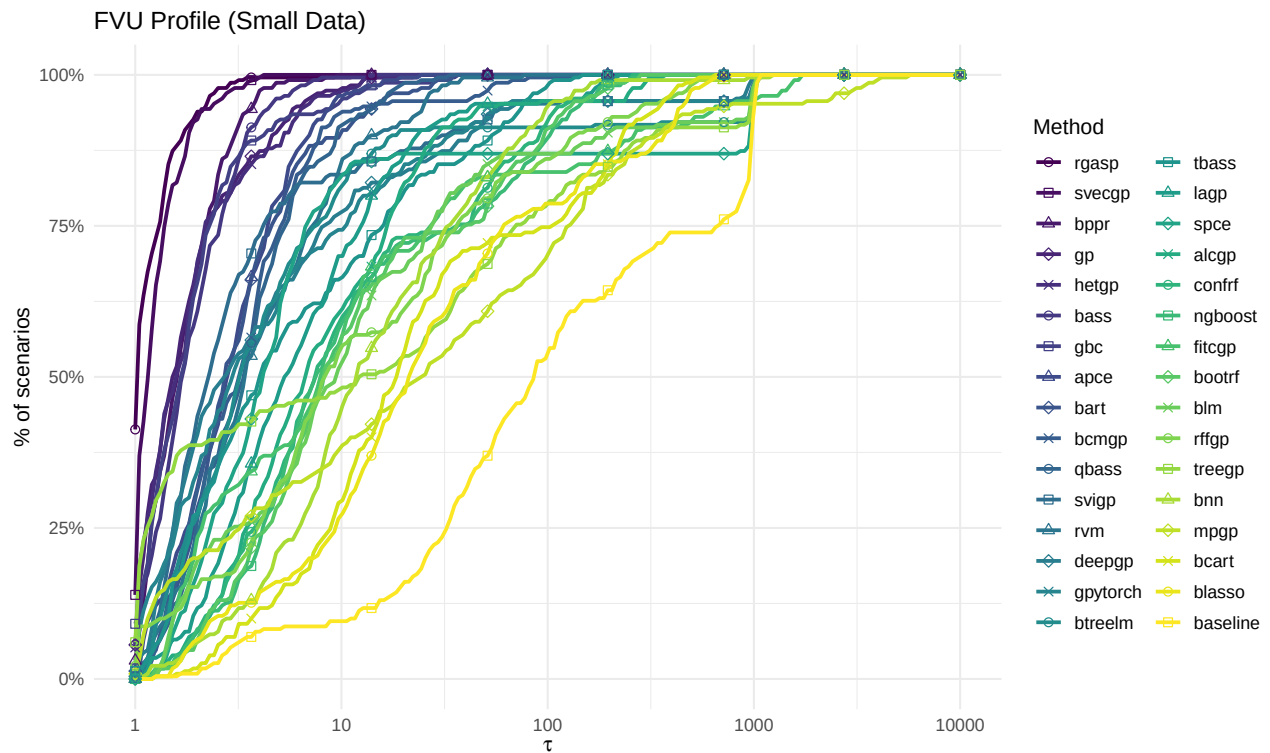
```



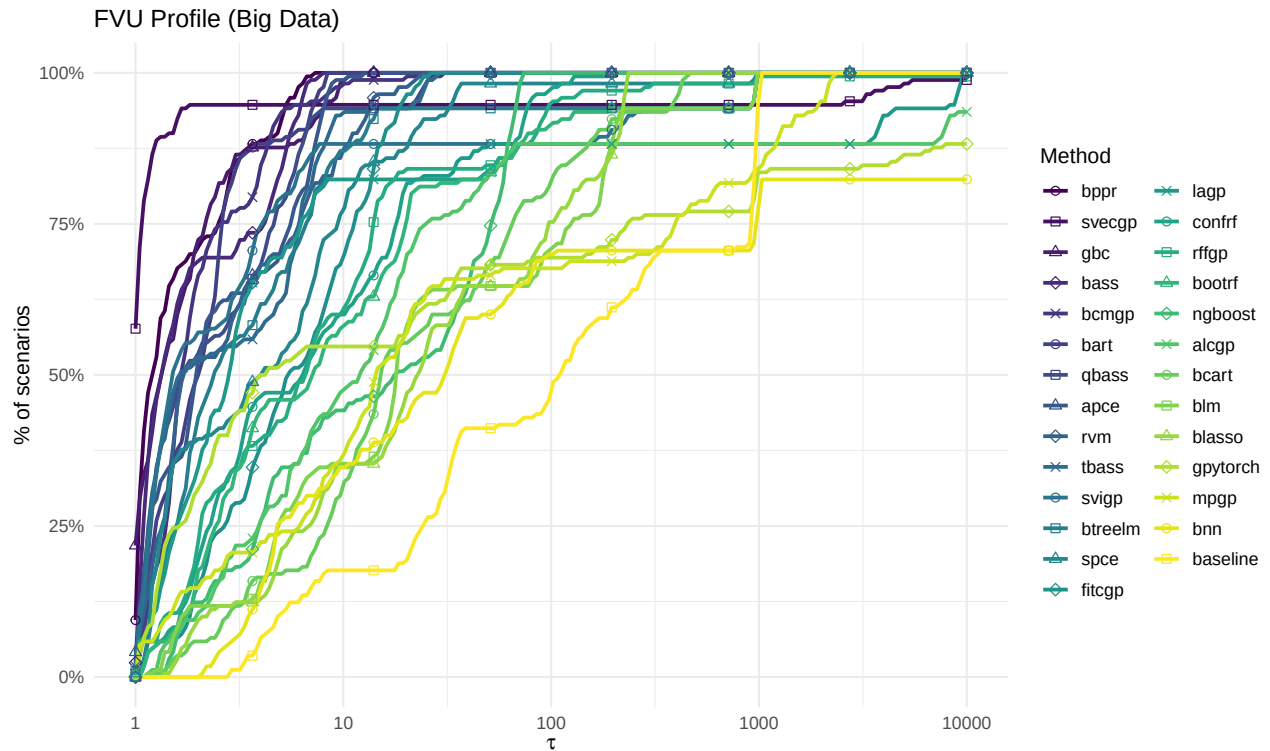




```
# Small data
title <- paste0("FVU Profile (Small Data)")
tmp <- filter_sim_study(duq_realddata,
  expr = n_train <= 1500)
p <- perfprofile_sim_study(tmp, metric="FVU",
  relative_epsilon = 0.001, tau_lim=c(1, 10000),
  title=title, color_scale = "viridis")
p <- p + ggplot2::theme_minimal(base_size=12)
plot(p)
```



```
# Large data
title <- paste0("FVU Profile (Big Data)")
tmp <- filter_sim_study(duq_realddata,
  expr = n_train > 1500)
p <- perfprofile_sim_study(tmp, metric="FVU",
  relative_epsilon = 0.001, tau_lim=c(1, 10000),
  title=title, color_scale = "viridis")
p <- p + ggplot2::theme_minimal(base_size=12)
plot(p)
```

5.1.3 FVU Heatmaps

We use a custom colorbar scale to make the figures more readable.

```
custom <- function(xx){
  log10(pmin(1, pmax(0.001, xx)))
}

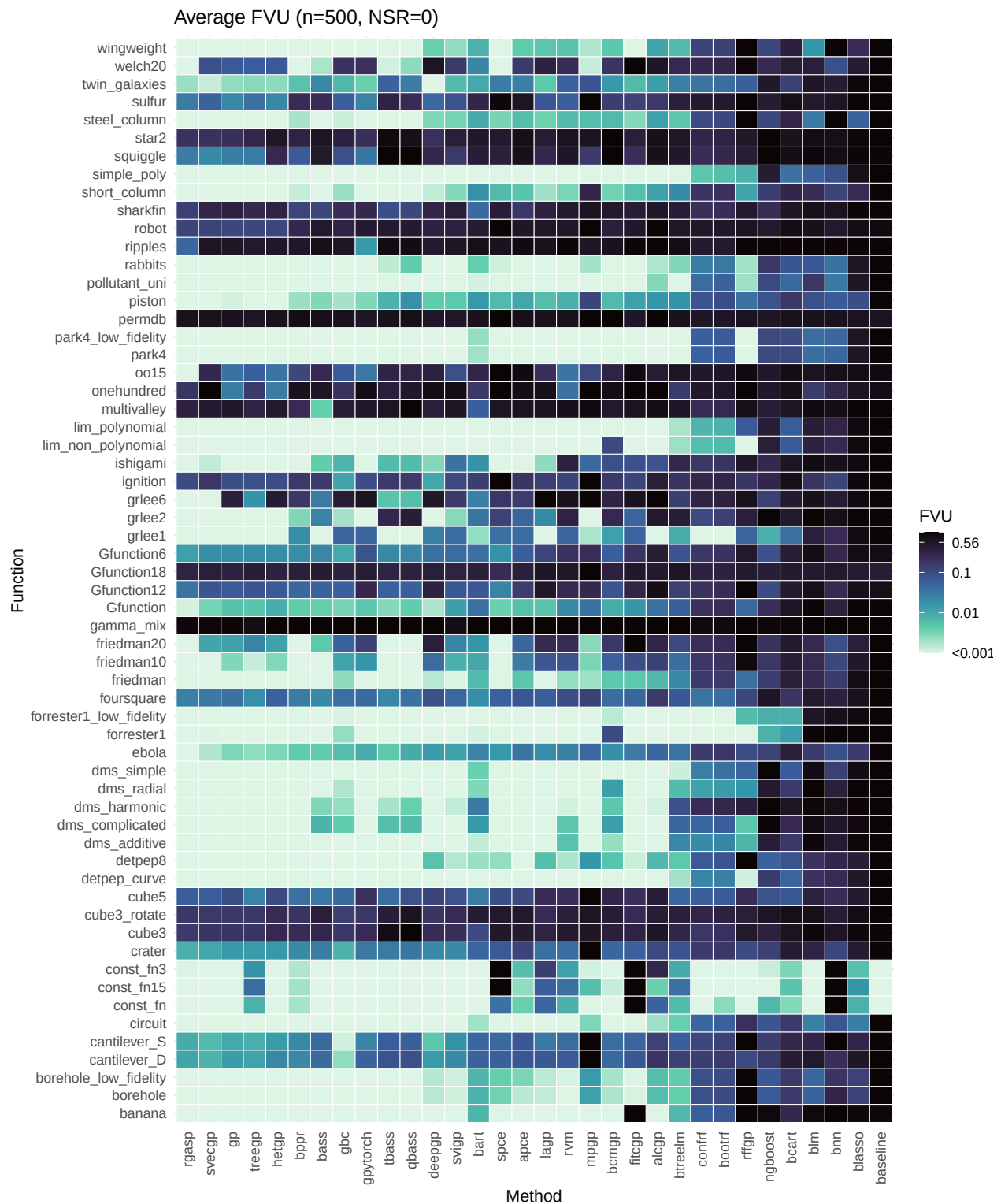
n_train_vec <- c(500, 500, 1000, 1000, 5000)
NSR_vec      <- c(0, 0.1, 0, 0.1, 0)
for(i in 1:5){
  title <- paste0("Average FVU (n=", n_train_vec[i], ", NSR=", NSR_vec[i], ")")
  tmp <- filter_sim_study(duq_testfuncs,
    n_train = n_train_vec[i], NSR = NSR_vec[i])
  p <- heatmap_sim_study(tmp, metric="FVU", y_scale_fun = custom,
    colorbar_labels = list(breaks=log10(c(0.001, 0.01, 0.1, 0.56, 1)),
      labels=c("<0.001", "0.01", "0.1", "0.56", "")),
    color_scale="mako",
    title=title)

  # Adjust sizing
  p <- p + ggplot2::theme_minimal(base_size=12) +
  ggplot2::theme(
    axis.text.x = ggplot2::element_text(
      angle = 90,
      vjust = 0.5,
      hjust = 1
    )
  )
}
```

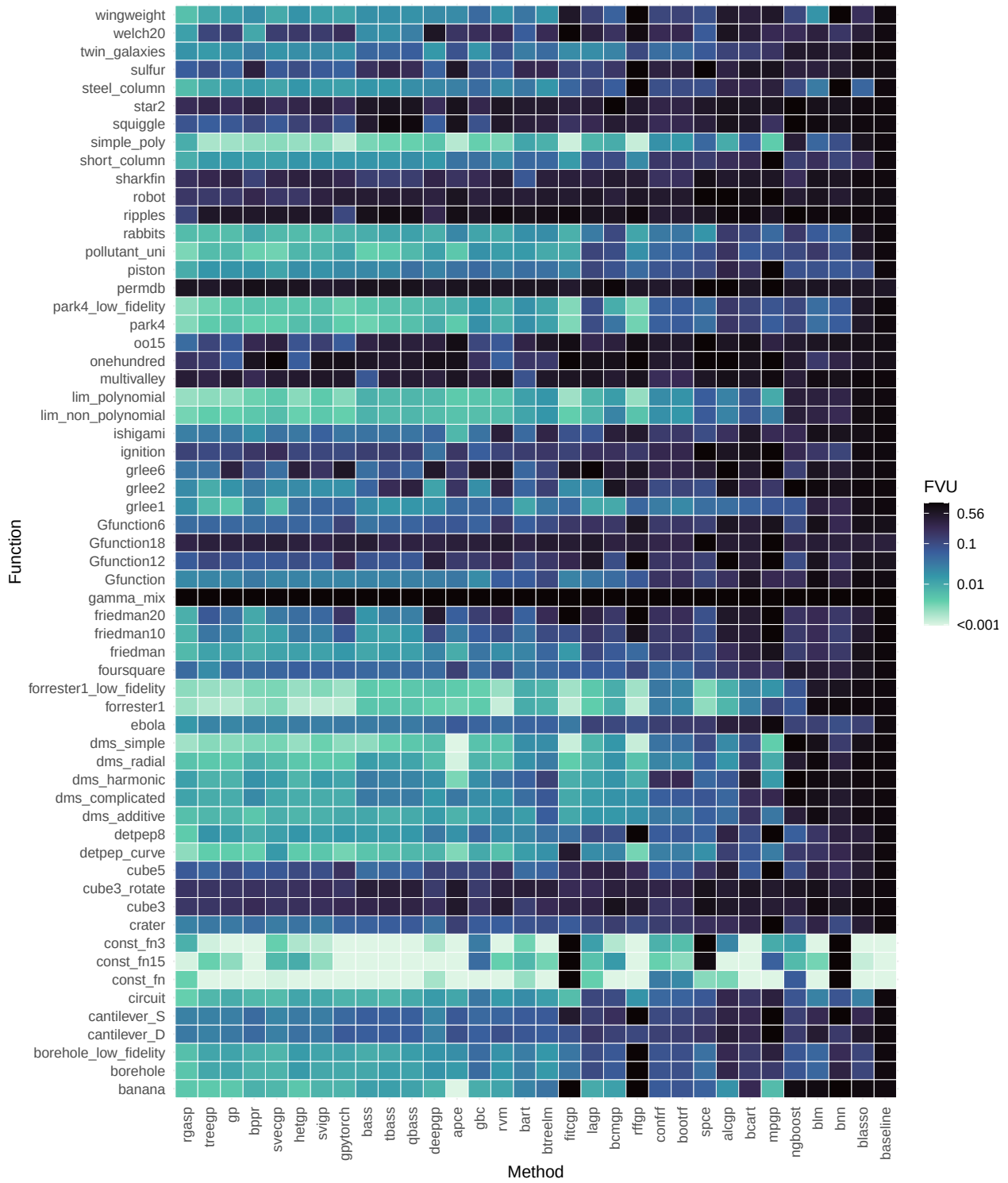
```

))
plot(p)
}

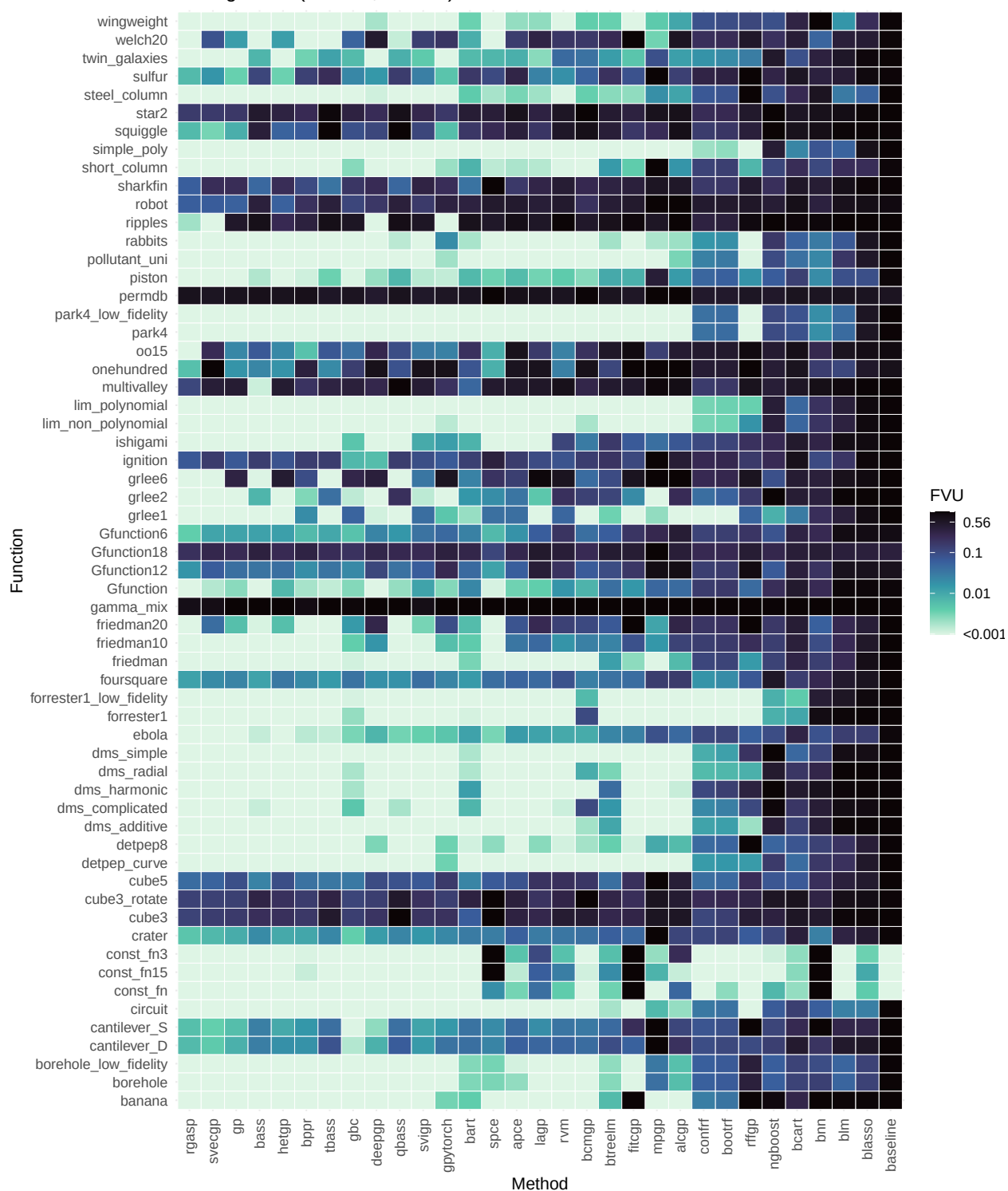
```

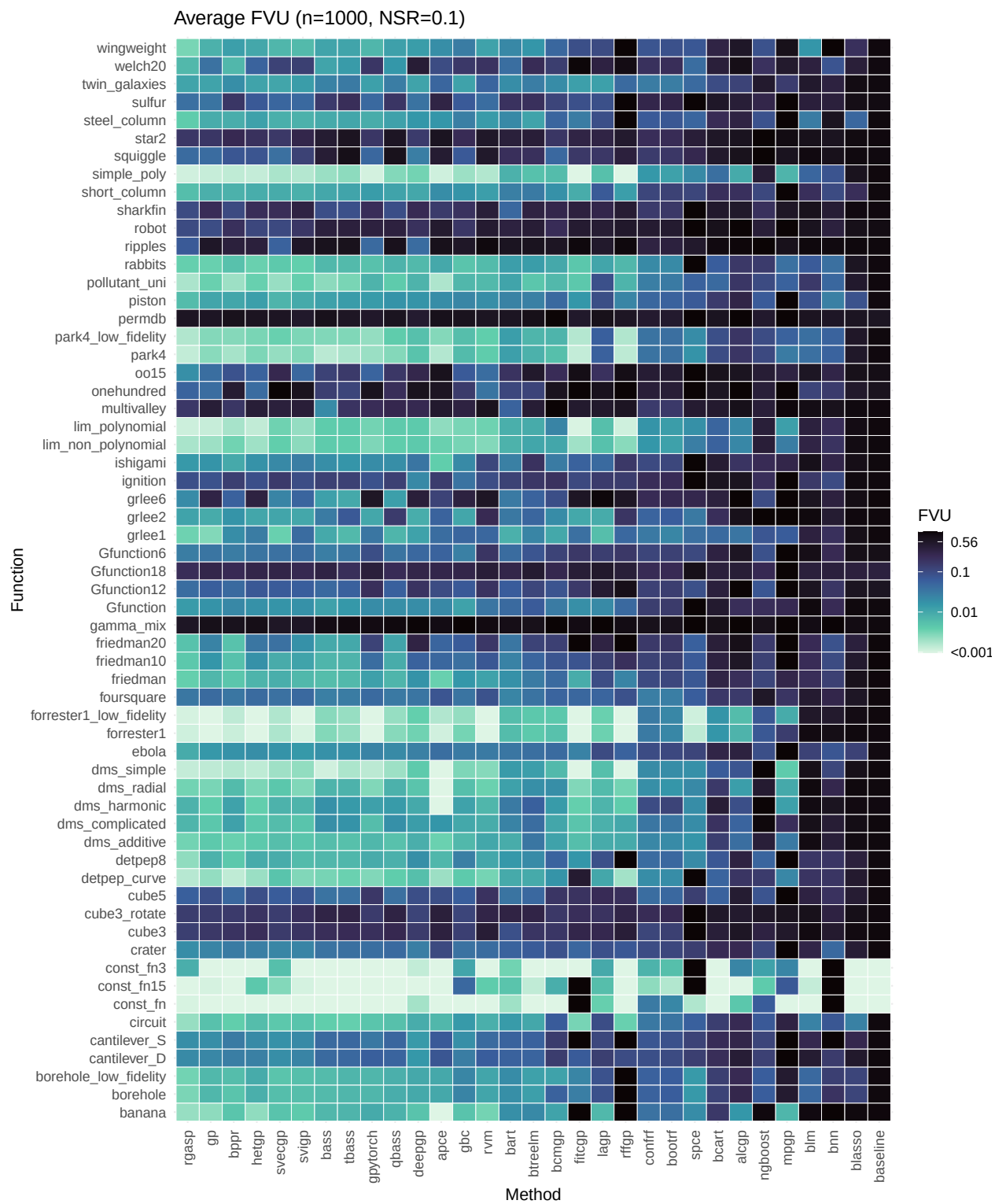


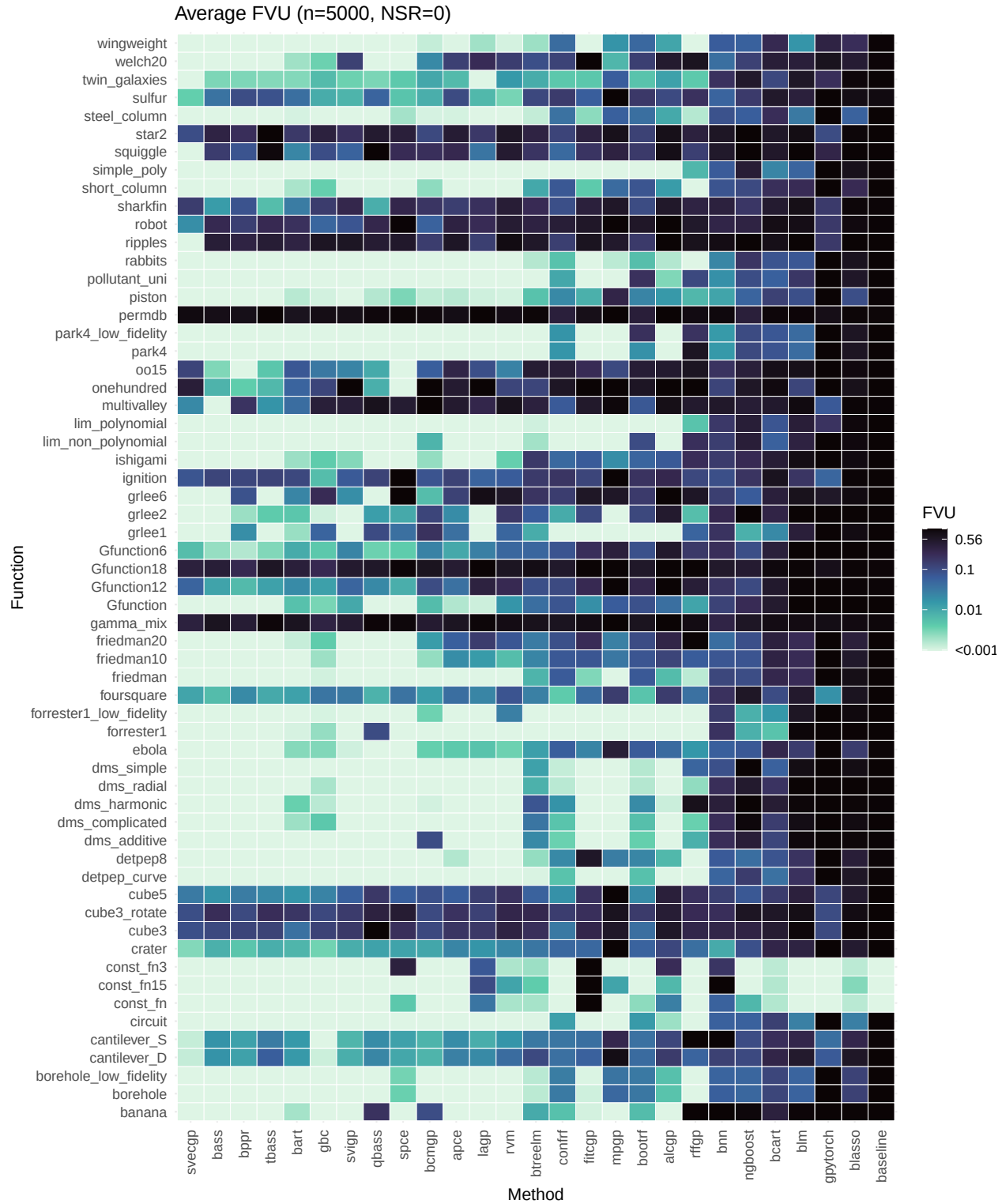
Average FVU (n=500, NSR=0.1)



Average FVU (n=1000, NSR=0)





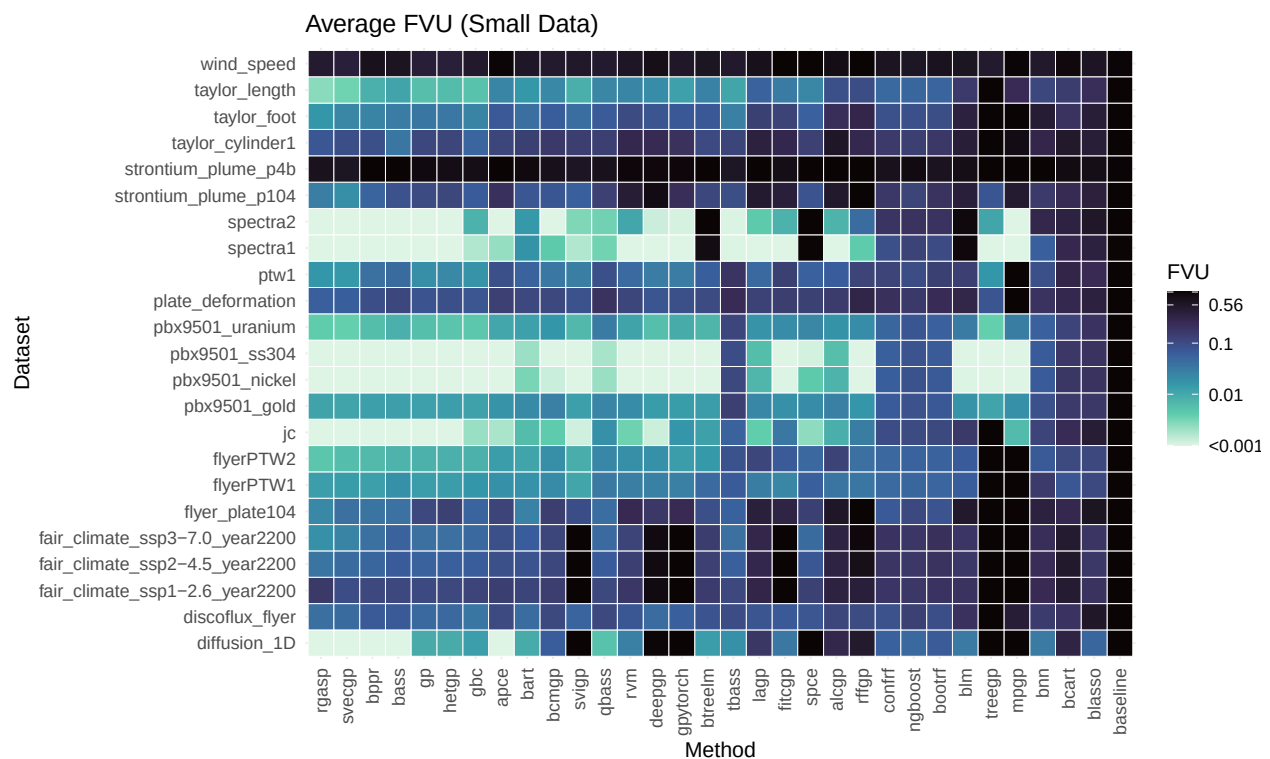


```
# Small data
title <- paste0("Average FVU (Small Data)")
tmp <- filter_sim_study(duq_realdata,
                        expr = n_train <= 1500)
p <- heatmap_sim_study(tmp, metric="FVU", y_scale_fun = custom,
```

```

        colorbar_labels = list(breaks=log10(c(0.001, 0.01, 0.1, 0.56, 1)),
                                labels=c("<0.001", "0.01", "0.1", "0.56", "")),
        color_scale="mako",
        title=title)
p <- p + ggplot2::theme_minimal(base_size=12) +
  ggplot2::theme(
    axis.text.x = ggplot2::element_text(
      angle = 90,
      vjust = 0.5,
      hjust = 1
    )
  )
plot(p)

```



```

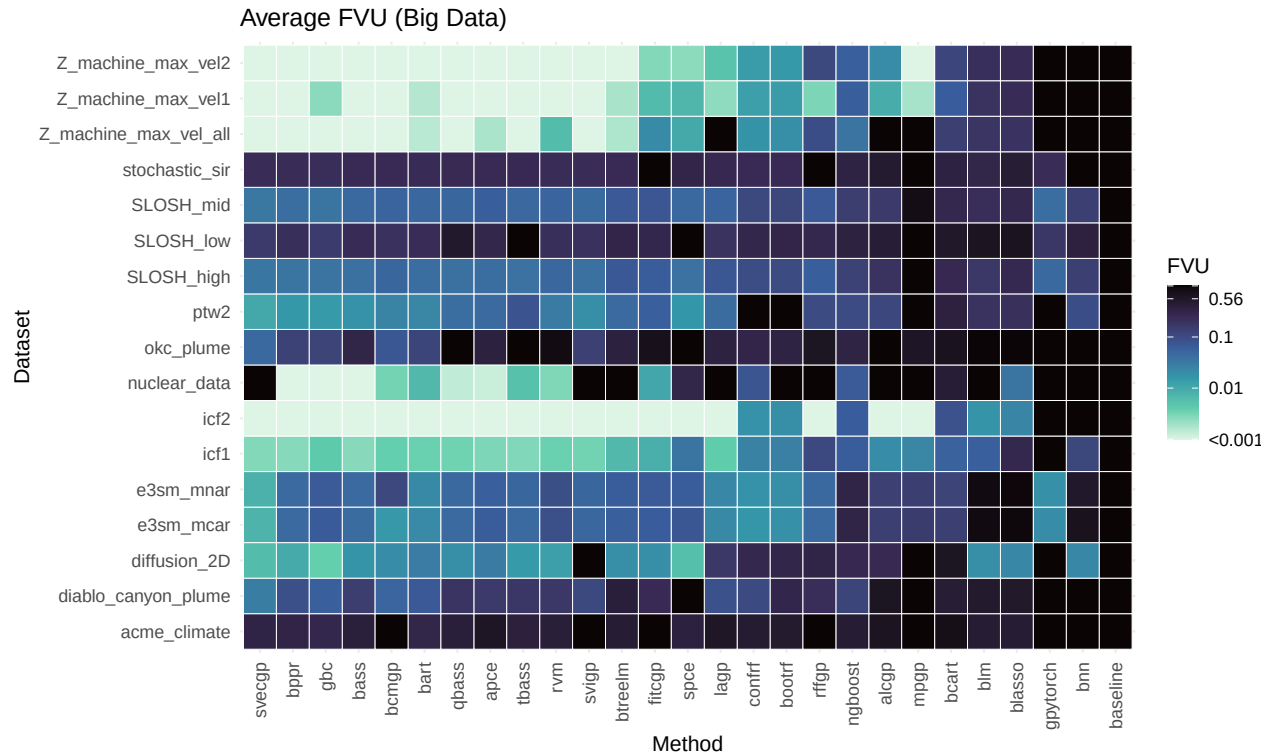
# Large data
title <- paste0("Average FVU (Big Data)")
tmp <- filter_sim_study(duq_realddata,
  expr = n_train > 1500)
p <- heatmap_sim_study(tmp, metric="FVU", y_scale_fun = custom,
  colorbar_labels = list(breaks=log10(c(0.001, 0.01, 0.1, 0.56, 1)),
    labels=c("<0.001", "0.01", "0.1", "0.56", "")),
  color_scale="mako",
  title=title)
p <- p + ggplot2::theme_minimal(base_size=12) +
  ggplot2::theme(
    axis.text.x = ggplot2::element_text(
      angle = 90,

```

```

    vjust = 0.5,
    hjust = 1
  ))
plot(p)

```



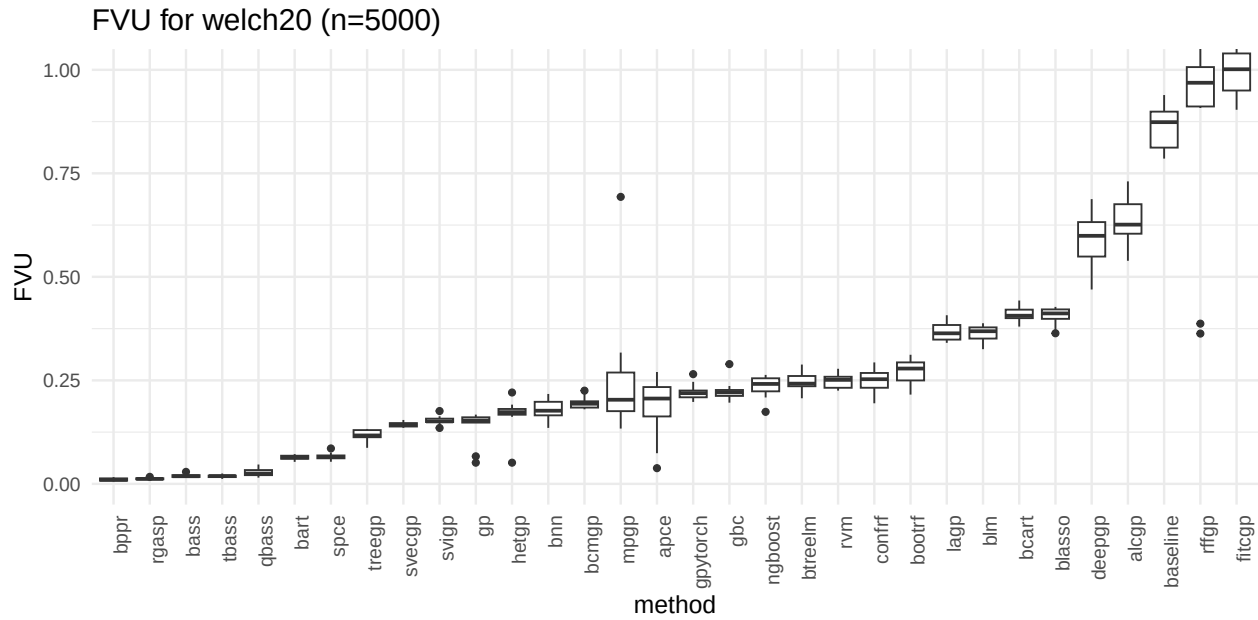
5.1.4 FVU Boxplots

Boxplots are a great way to summarize a specific simulation scenario. There are thousands of possible boxplots one could make from this data, here's an example.

```

tmp <- filter_sim_study(duq_testfuncs, id = "welch20",
  n_train=500, NSR=0.1)
boxplots_sim_study(tmp, metric="FVU",
  ylim=c(0, 1), title="FVU for welch20 (n=5000)")

```

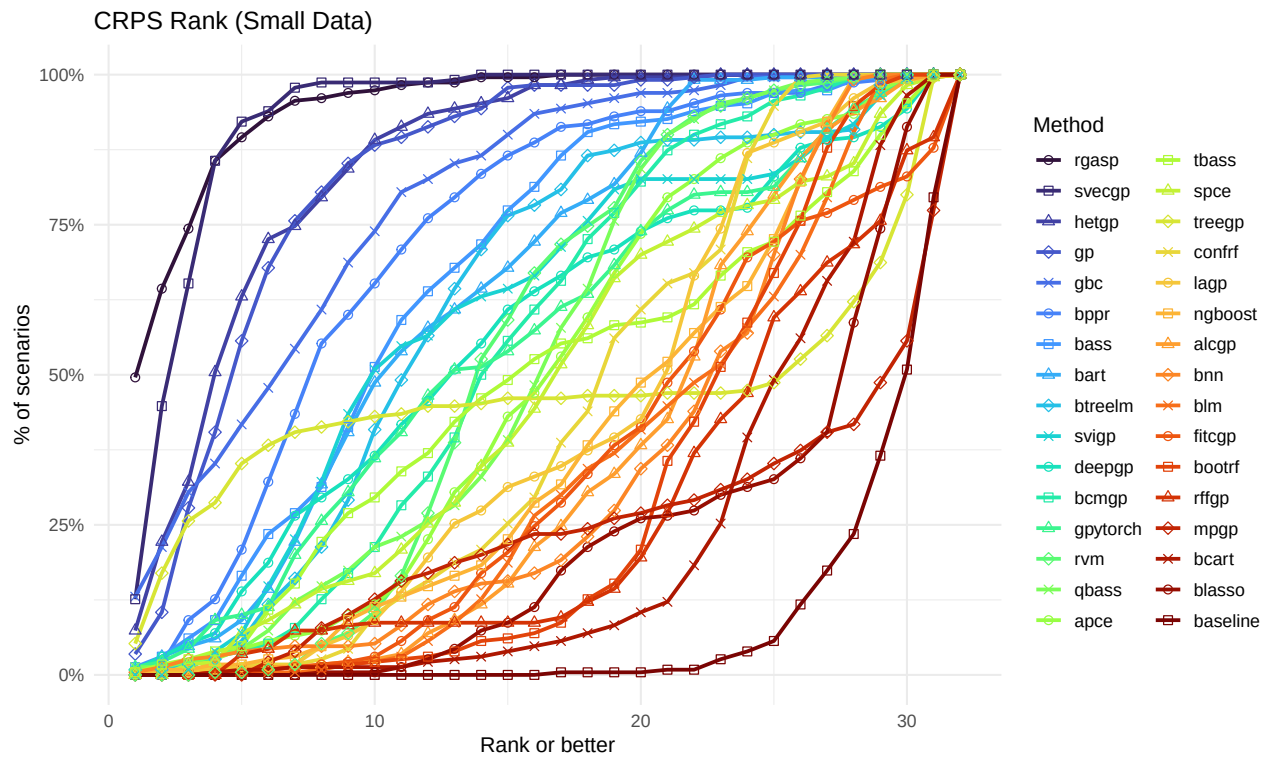



5.2 Continuous Ranked Probability Scores

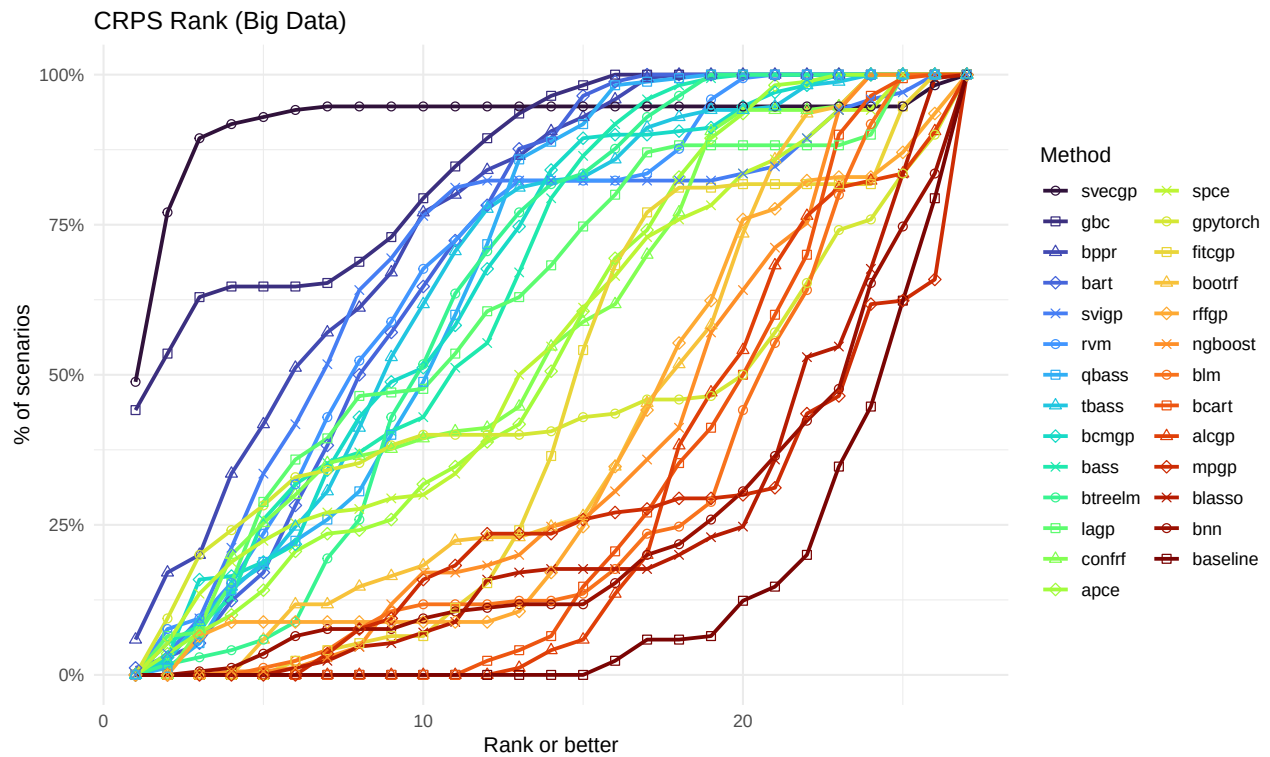
Continuous ranked probability scores (CRPS) was the primary metric used in the main manuscript, and so we won't rehash all of that here. Here we show some key figures not given in the main manuscript: (i) CRPS rankplots for the real datasets and (ii) CRPS performance profiles for the test functions.

5.2.1 CRPS Rankplots (data only)

```
# Small data
title <- paste0("CRPS Rank (Small Data)")
tmp <- filter_sim_study(duq_realdata,
                        expr = n_train <= 1500)
p <- rankplot_sim_study(tmp, metric="CRPS", title=title)
p <- p + ggplot2::theme_minimal(base_size=12)
plot(p)
```



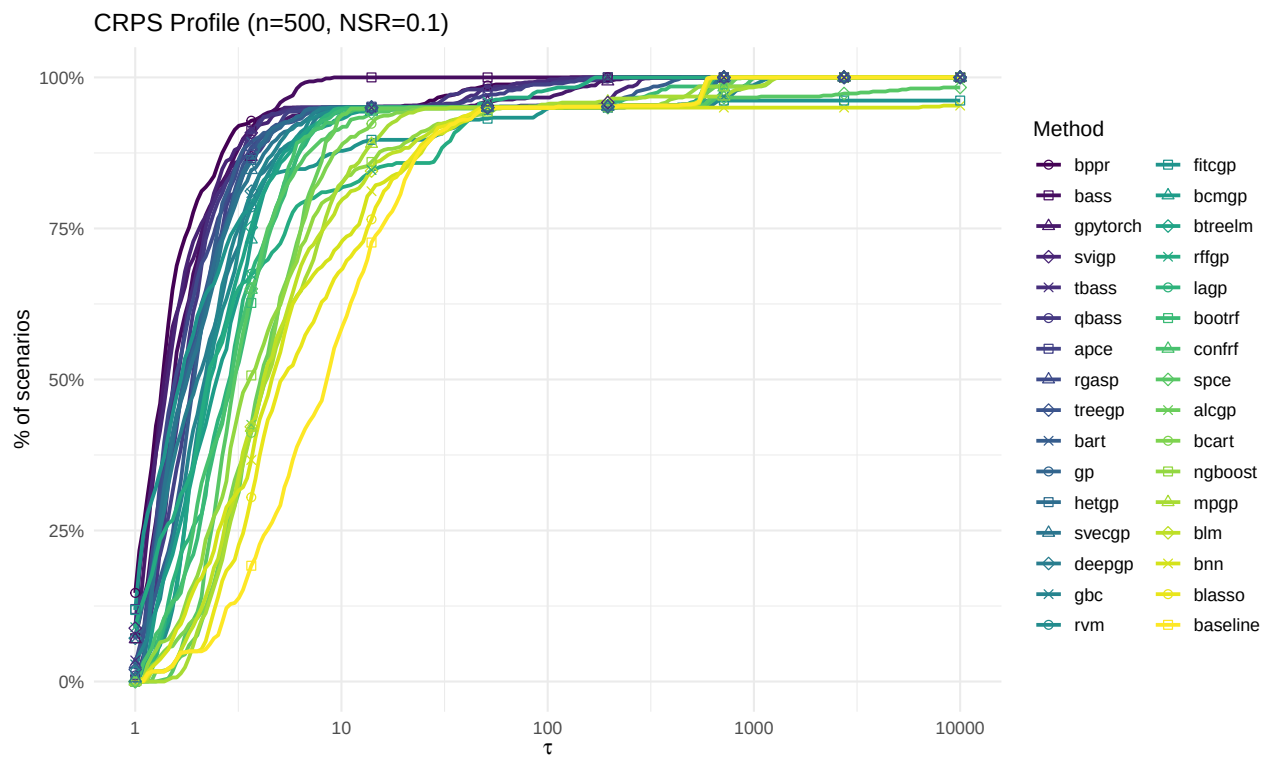
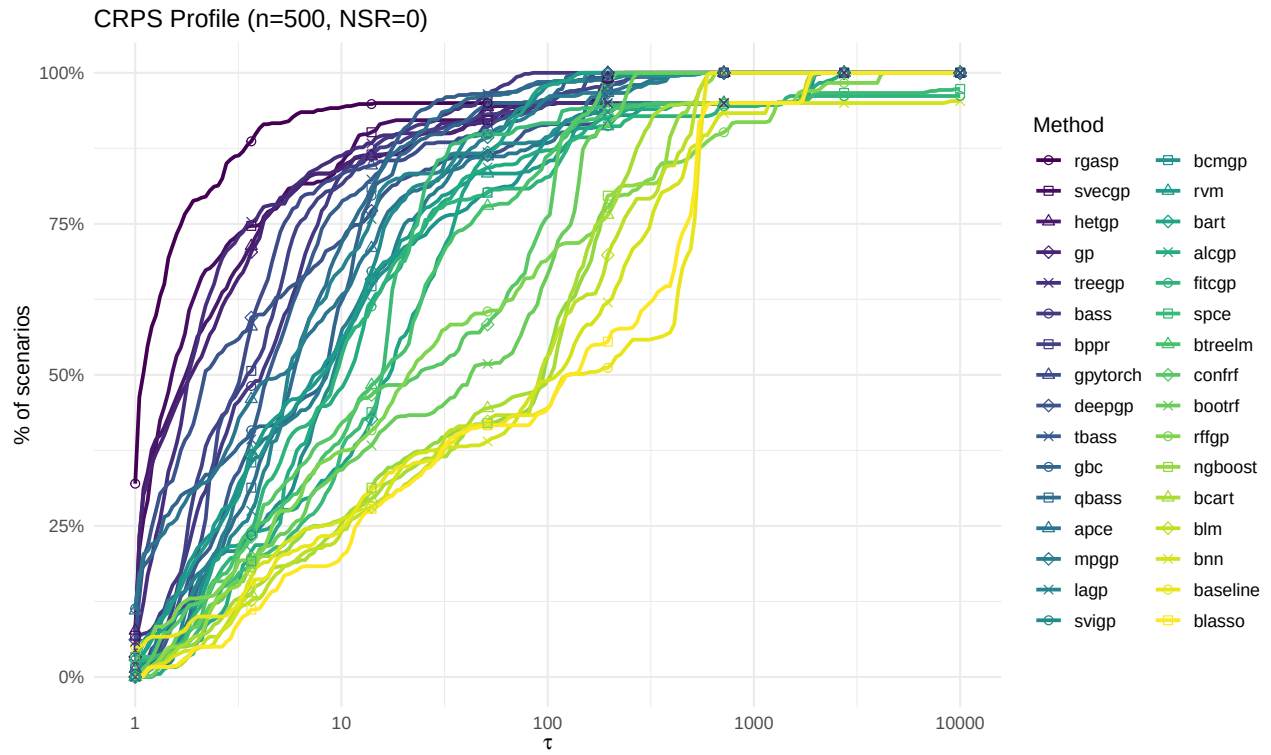
```
# Large data
title <- paste0("CRPS Rank (Big Data)")
tmp <- filter_sim_study(duq_realdata,
                        expr = n_train > 1500)
p <- rankplot_sim_study(tmp, metric="CRPS", title=title)
p <- p + ggplot2::theme_minimal(base_size=12)
plot(p)
```

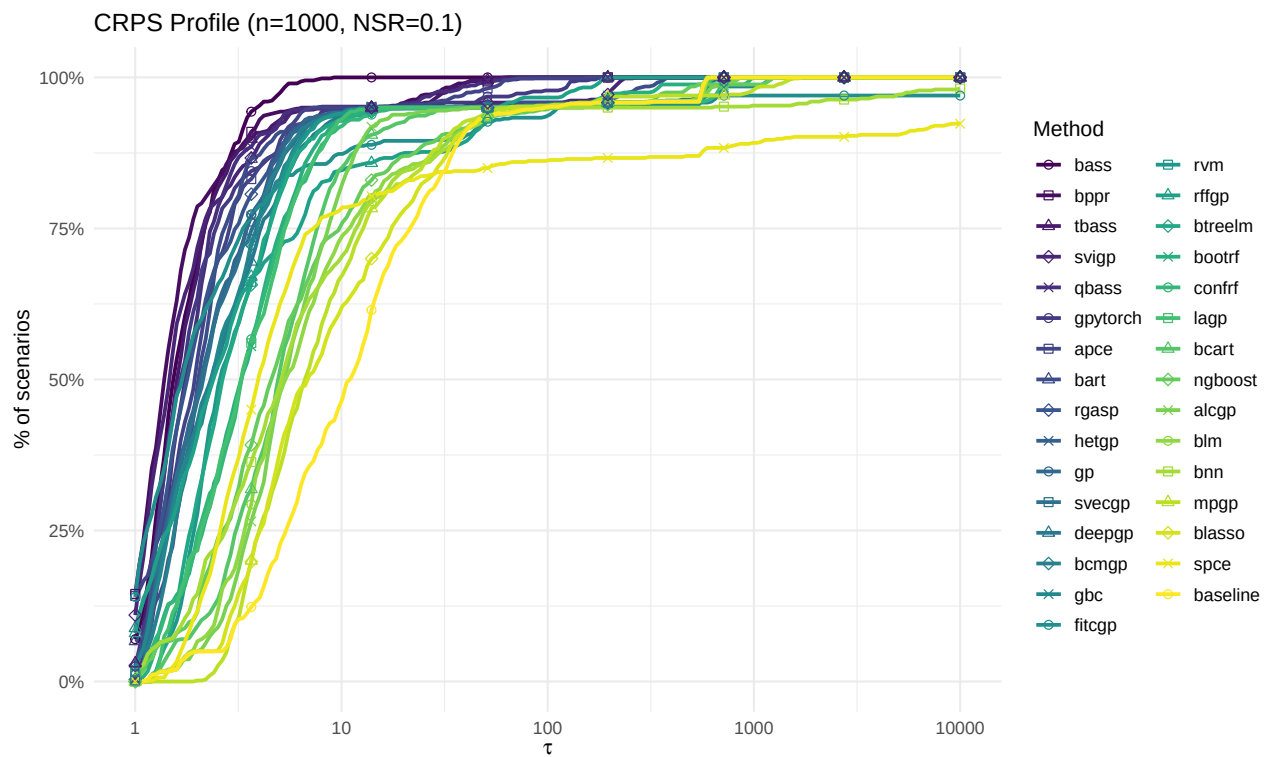
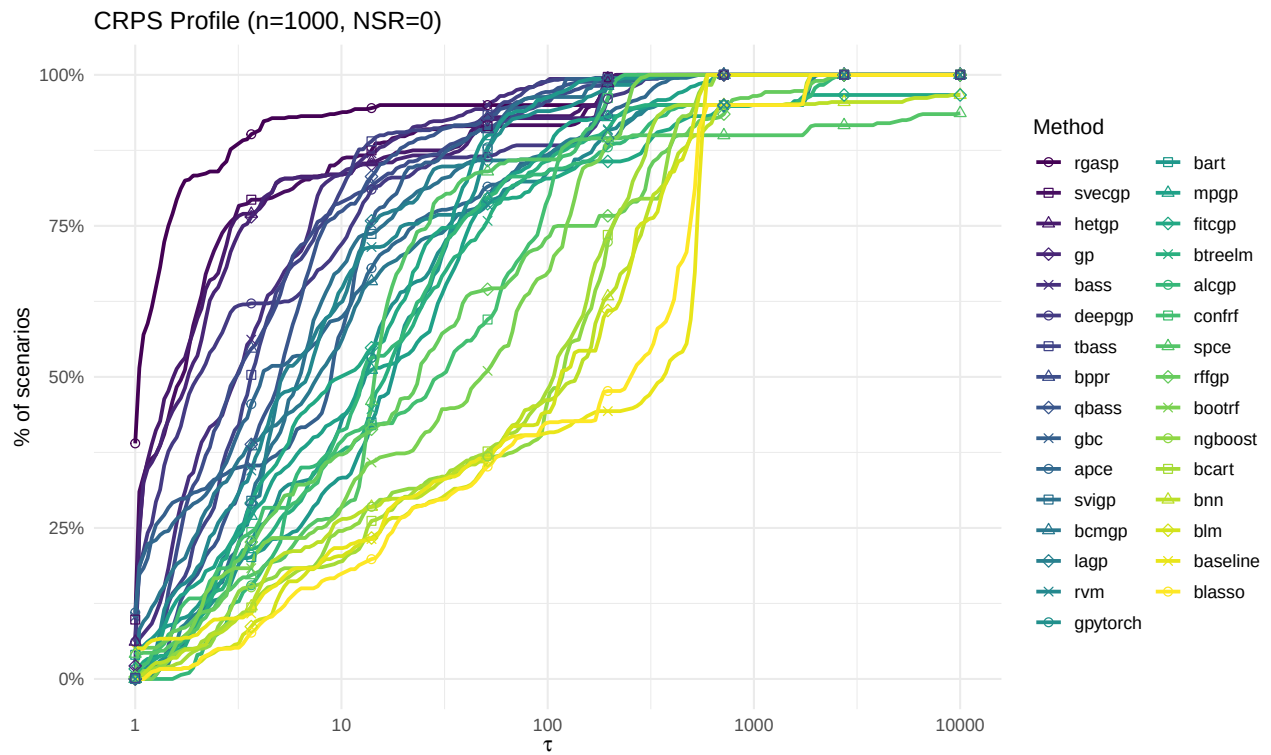


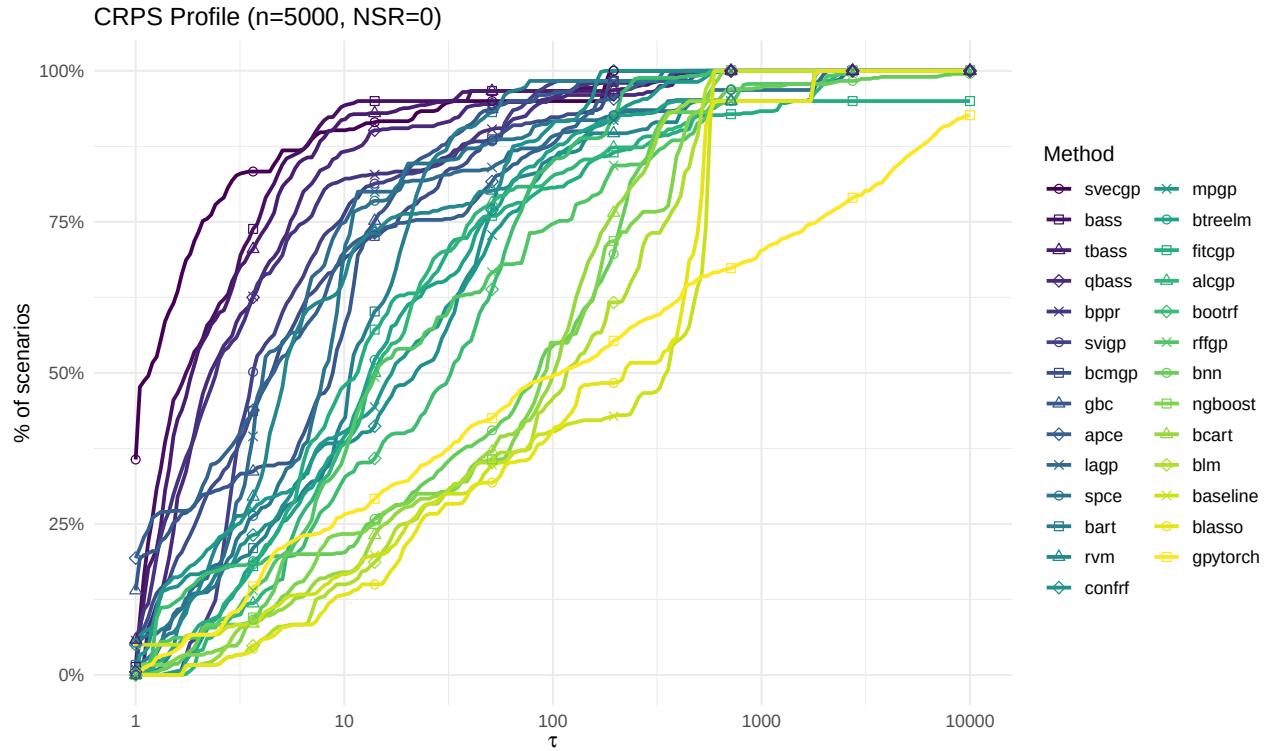
5.2.2 CRPS Performance Profiles (testfuncs only)

```
n_train_vec <- c(500, 500, 1000, 1000, 5000)
NSR_vec      <- c(0, 0.1, 0, 0.1, 0)
for(i in 1:5){
  title <- paste0("CRPS Profile (n=", n_train_vec[i], ", NSR=", NSR_vec[i], ")")
  tmp <- filter_sim_study(duq_testfuncs,
                          n_train = n_train_vec[i], NSR = NSR_vec[i])
  p <- perfprofile_sim_study(tmp, metric="CRPS",
                            relative_epsilon = 0.001, tau_lim=c(1, 10000),
                            title=title, color_scale = "viridis")

  # Adjust sizing
  p <- p + ggplot2::theme_minimal(base_size=12)
  plot(p)
}
```







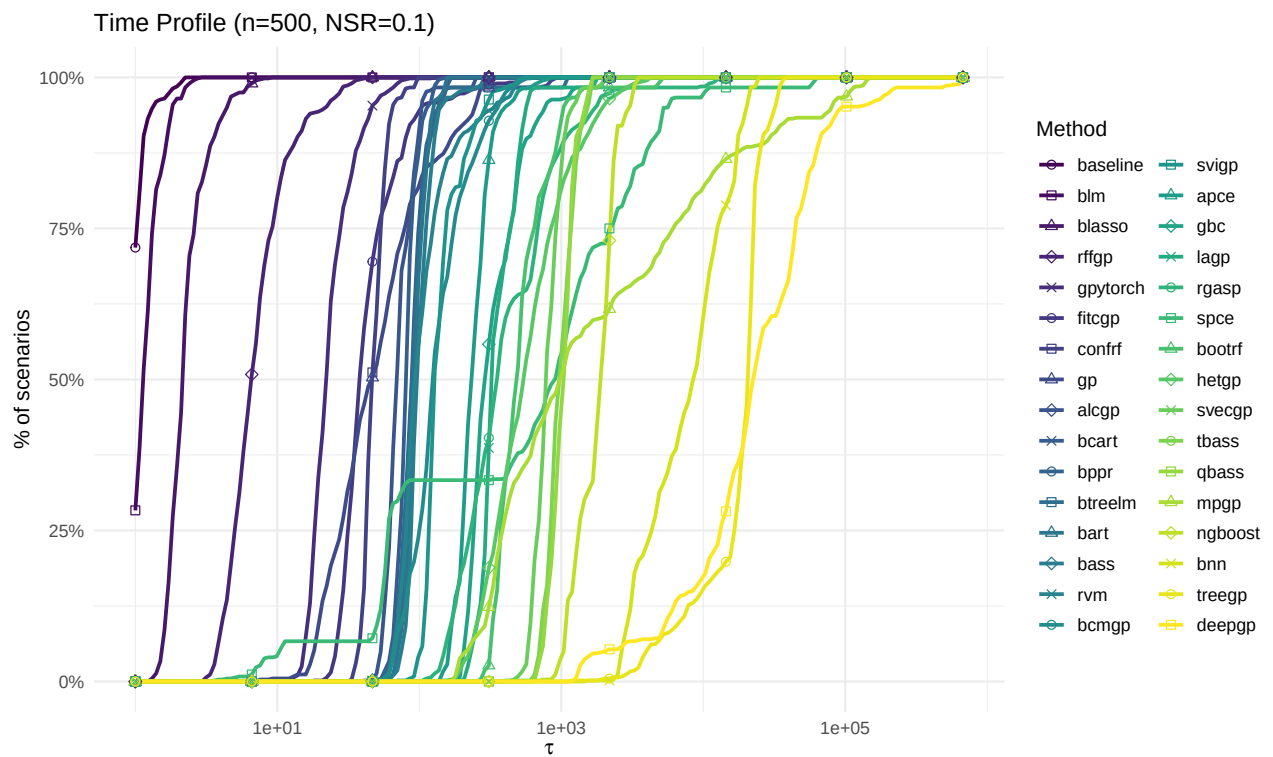
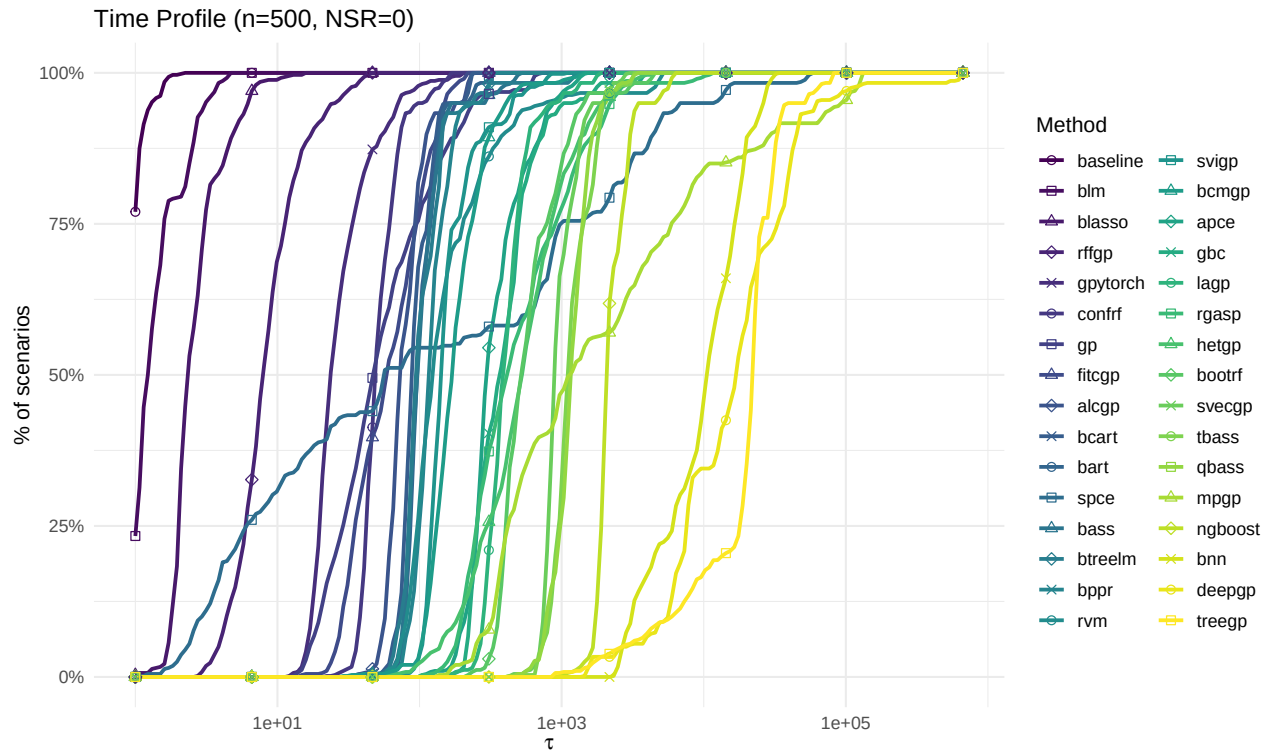
5.3 Computation Time

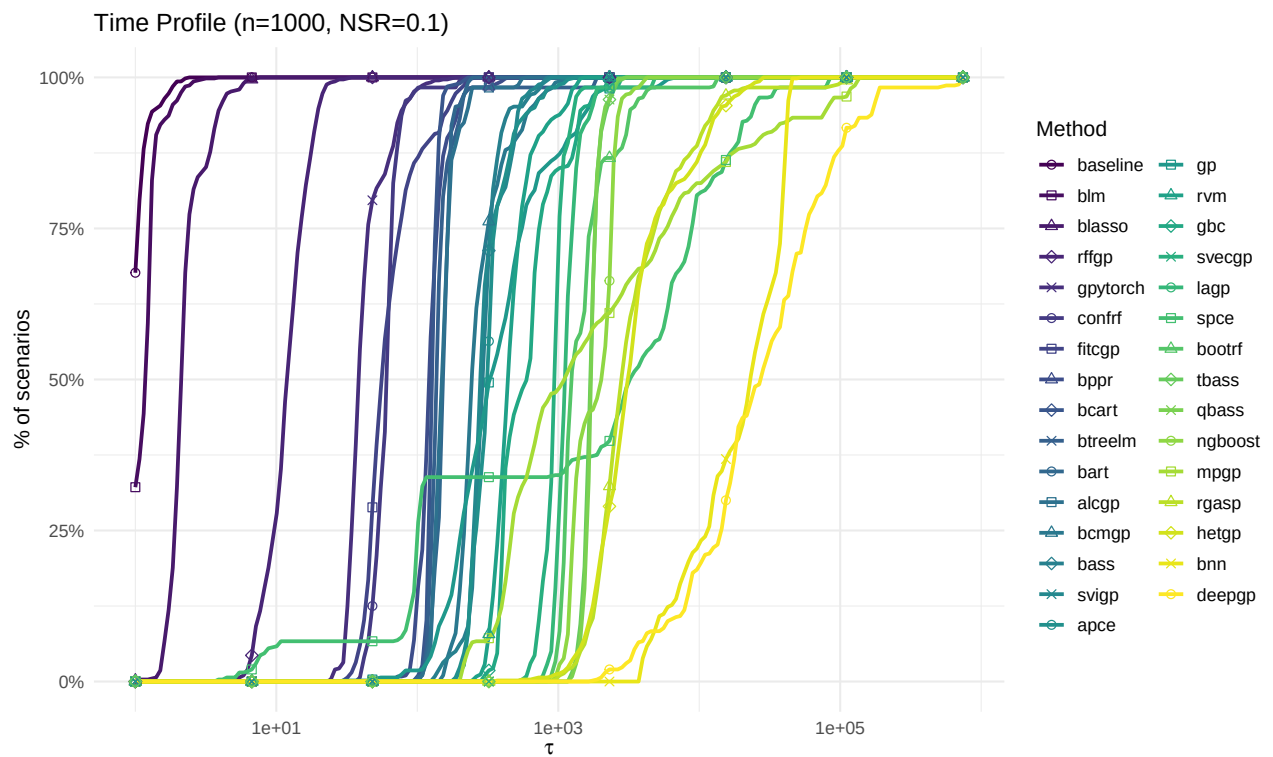
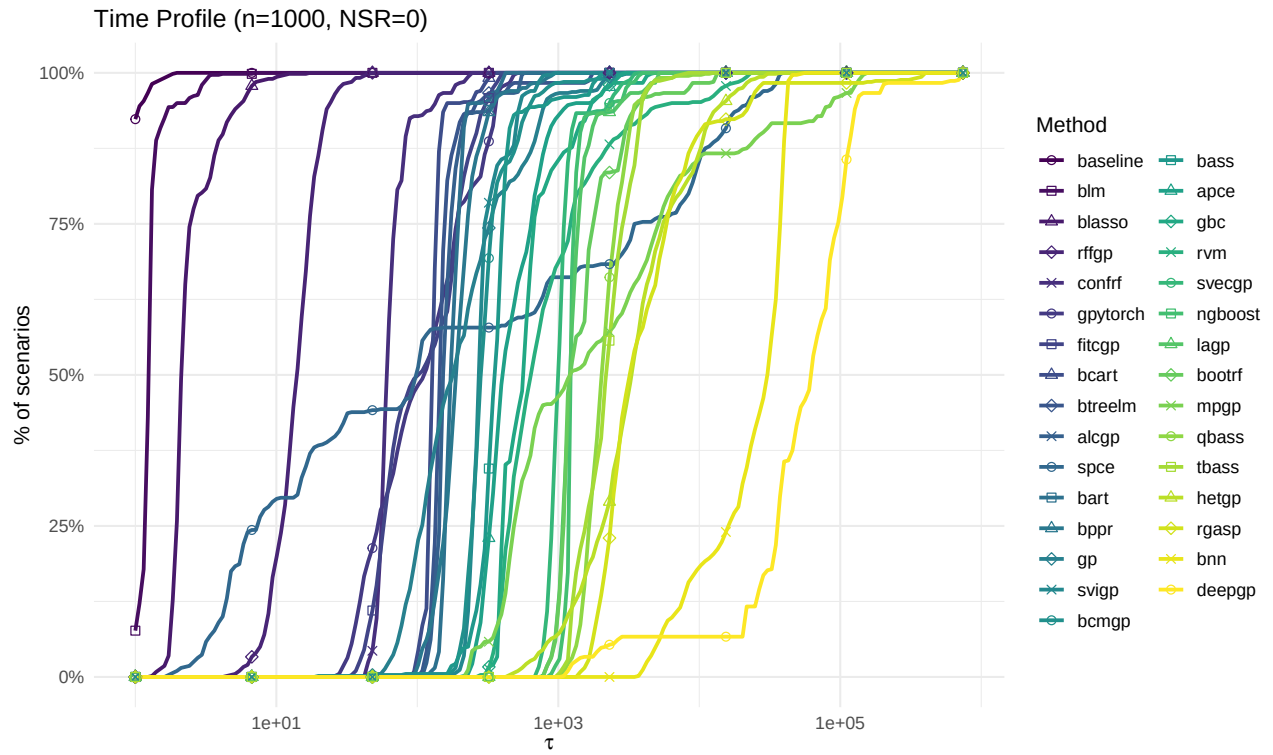
The `duqling` package usually times both training and prediction splits, but for this paper we report just the total computation time, because several of the surrogate modeling implementations cannot be split easily.

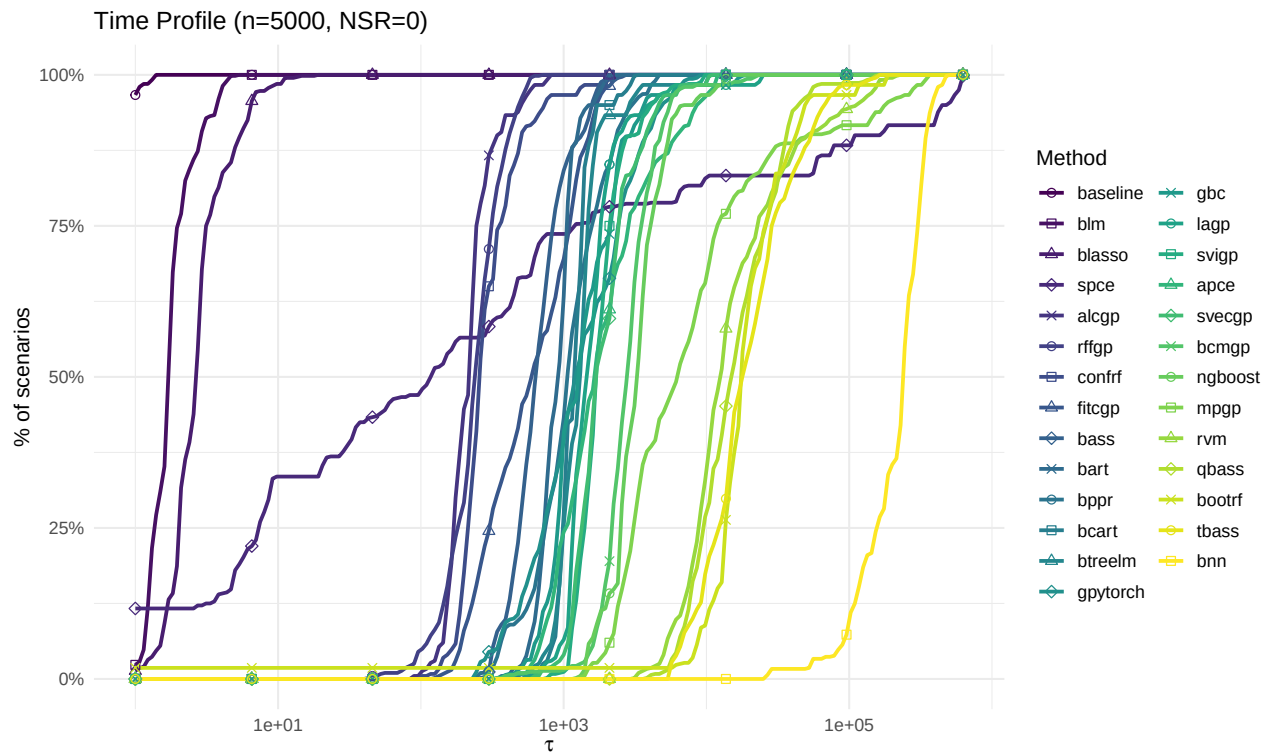
5.3.1 Time Performance Profiles

```
# Synthetic test functions (5 cases)
n_train_vec <- c(500, 500, 1000, 1000, 5000)
NSR_vec     <- c(0, 0.1, 0, 0.1, 0)

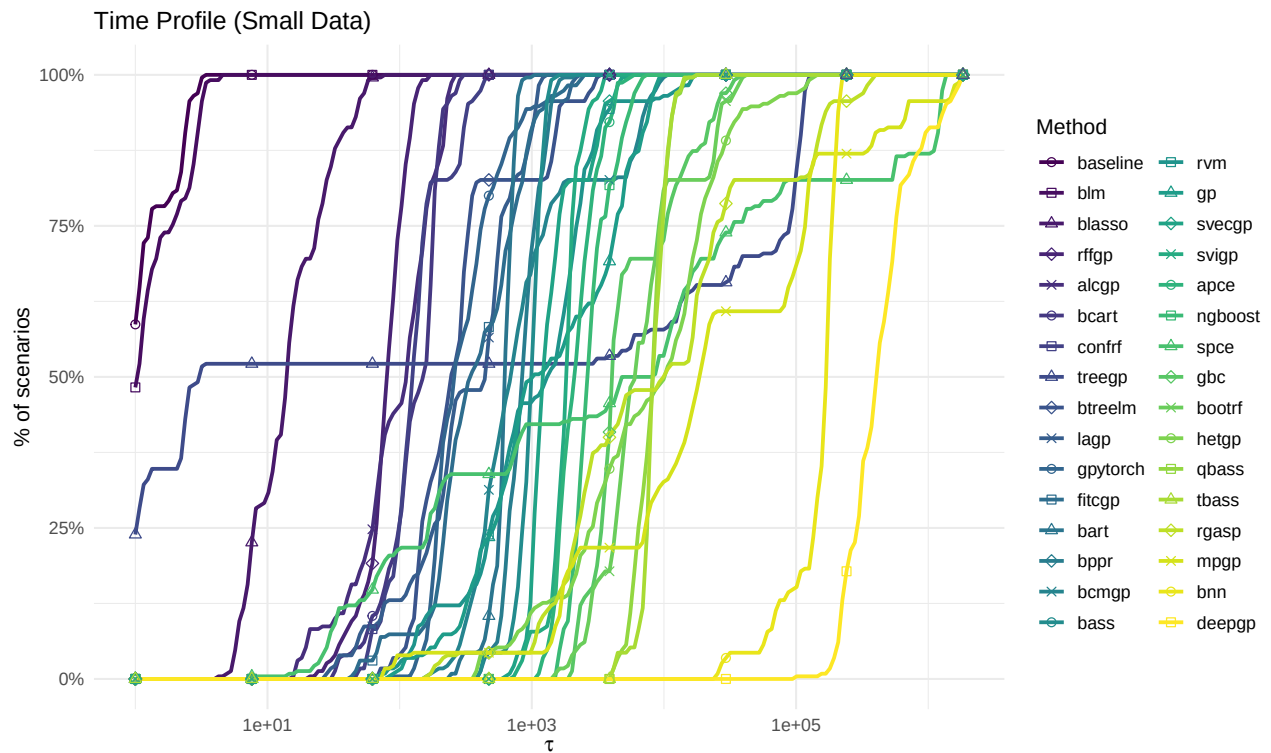
for(i in 1:5){
  title <- paste0("Time Profile (n=", n_train_vec[i], ", NSR=", NSR_vec[i], ")")
  tmp <- filter_sim_study(duq_testfuncs,
                          n_train = n_train_vec[i], NSR = NSR_vec[i])
  p <- perfprofile_sim_study(tmp, metric="time",
                            title=title, color_scale = "viridis")
  p <- p + ggplot2::theme_minimal(base_size=12)
  plot(p)
}
```



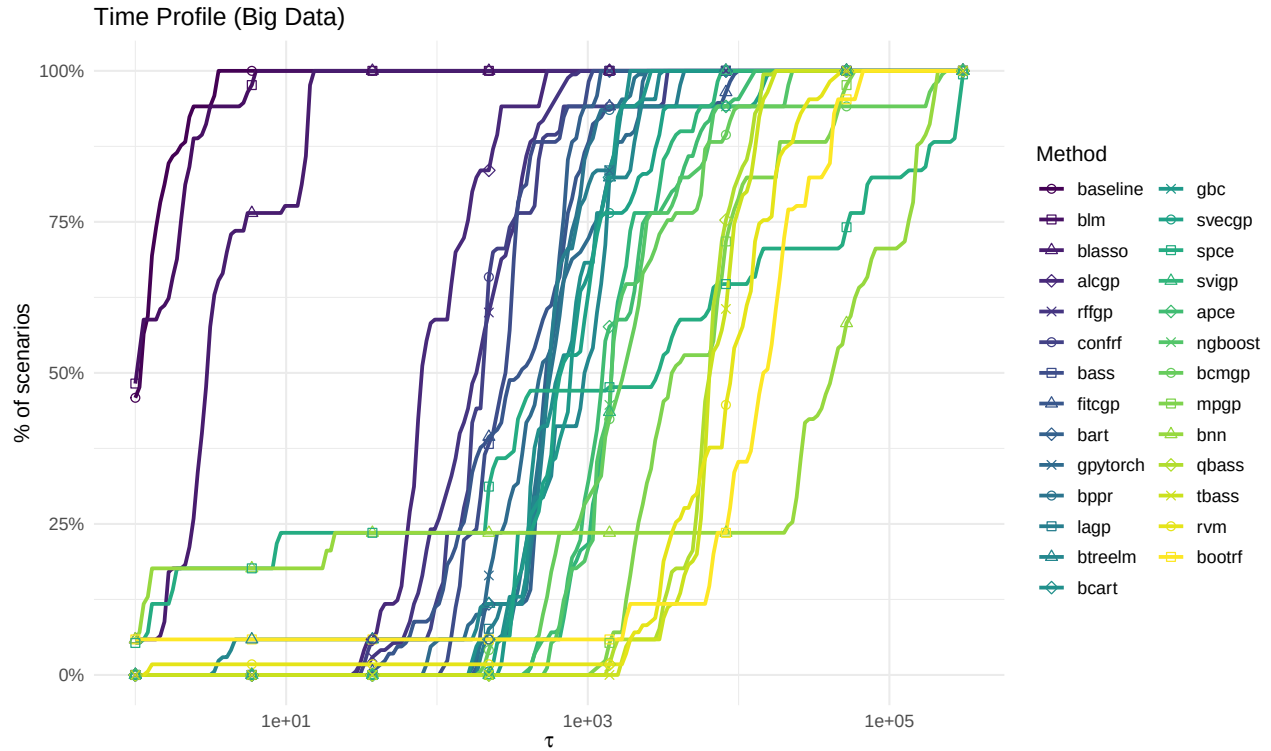




```
# Small data
title <- "Time Profile (Small Data)"
tmp <- filter_sim_study(duq_realddata, expr = n_train <= 1500)
p <- perfprofile_sim_study(tmp, metric="time",
                           title=title, color_scale = "viridis")
p <- p + ggplot2::theme_minimal(base_size=12)
plot(p)
```



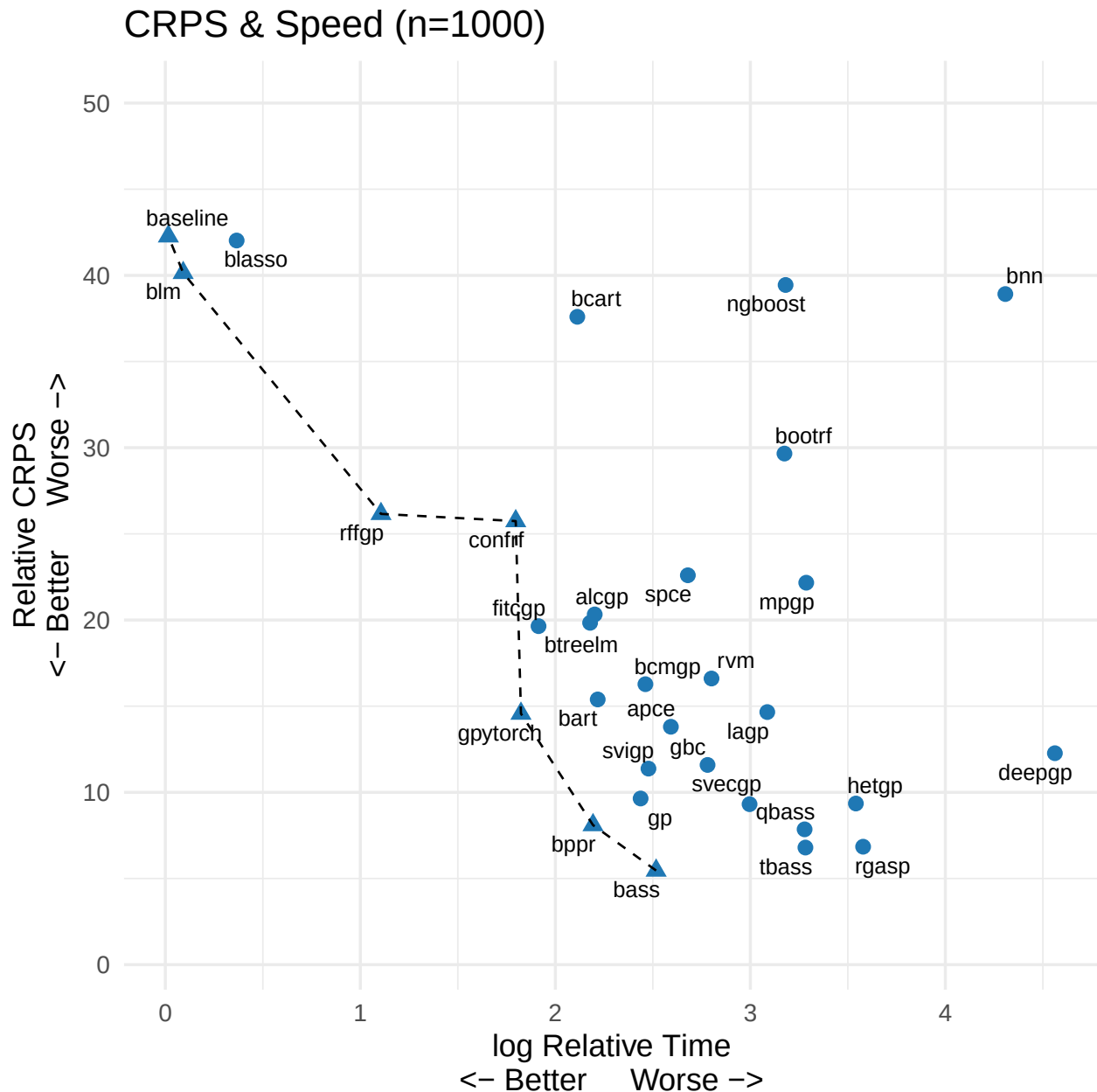
```
# Large data
title <- "Time Profile (Big Data)"
tmp <- filter_sim_study(duq_realdata, expr = n_train > 1500)
p <- perfprofile_sim_study(tmp, metric="time",
                           title=title, color_scale = "viridis")
p <- p + ggplot2::theme_minimal(base_size=12)
plot(p)
```



5.3.2 Time Paretoplots

Finally, we provide an example of how the paretoplots in the main paper were created (with an internal call to `duqling::relativize_sim_study`).

```
# This only filters on sample size, not NSR
tmp <- filter_sim_study(duq_testfuncs, n_train=1000)
paretoplot_sim_study(tmp, metric=c("CRPS_rel", "time_rel_log"),
  epsilon=c(0.001, 0), upper_bound=c(100, Inf),
  show_legend = FALSE, title="CRPS & Speed (n=1000)",
  ylim=c(1, 50))
```



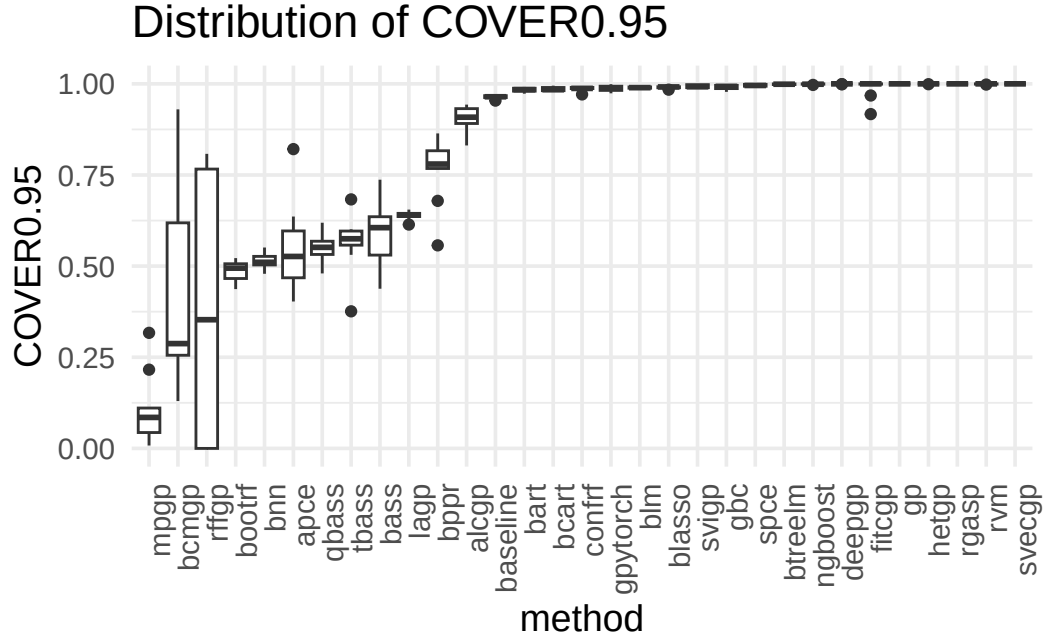
5.4 Other Metrics

The analyses presented in the main paper focus primarily on CRPS, because it is a proper scoring rule for predictions with uncertainty, but the `duqling` framework is flexible and supports a wide range of alternative metrics. In many uncertainty quantification applications, practitioners are interested in properties of predictive intervals, such as coverage or interval width, in addition to overall predictive accuracy.

Coverage is one of the most commonly reported diagnostics, but it is not a proper scoring rule and does not account for the width of the interval.

```
tmp <- filter_sim_study(duq_testfuncs,
  id="borehole",
```

```
n_train=1000, NSR=0.1)
boxplots_sim_study(tmp, metric="COVER0.95")
```



For a given confidence level, mean interval scores (MIS) are a proper scoring rule that factor in both accuracy and precision.

$$\text{MIS}_{1-\alpha} = \frac{1}{n_{\text{test}}} \sum_{i=1}^{n_{\text{test}}} \left[(U_i - L_i) + \frac{2}{\alpha} (L_i - y_{\text{test},i}) \mathbf{1}\{y_{\text{test},i} < L_i\} + \frac{2}{\alpha} (y_{\text{test},i} - U_i) \mathbf{1}\{y_{\text{test},i} > U_i\} \right],$$

while the current version of `duqling` does compute these scores, they were omitted from the original simulation study for simplicity.

One simple metric due to [25] that can be recovered, is the integrated absolute error across confidence levels $1 - \alpha$, defined as

$$\text{IAE-}\alpha = \int_0^1 |C_\alpha - \alpha| d\alpha,$$

where C_α is the empirical coverage for confidence level α . To estimate this accurately in `duqling`, we should run with a long list of confidence levels using the `conf_level` argument in `run_sim_study()` and `run_sim_study_data()`. But it can be interesting to look at the estimate obtained using the confidence levels run by the default settings. In this case, we approximate the integral by a simple discrete average over the confidence levels used in the simulation, i.e., by averaging the absolute differences between empirical and nominal coverage at each level.

$$\hat{\text{IAE}} = \frac{1}{K} \sum_{k=1}^K |\hat{C}_{\alpha_k} - \alpha_k|.$$

We can create this new column using the package's `mutate_sim_study()` function.

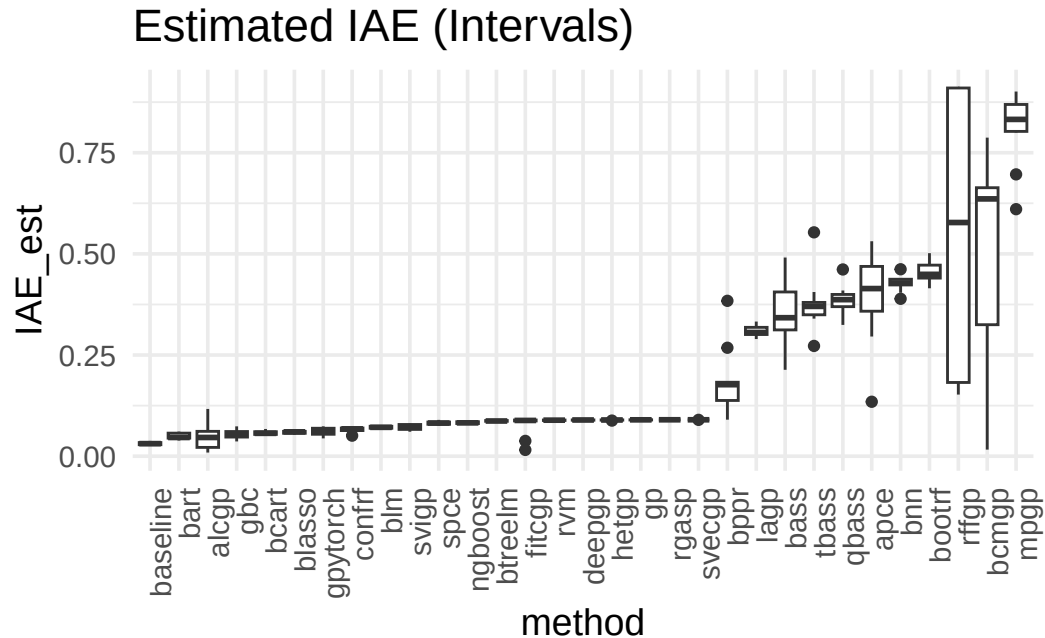
```
tmp_IAE <- mutate_sim_study(tmp,
  IAE_est = 0.25 * abs(COVER0.8 - 0.8) +
    0.25 * abs(COVER0.9 - 0.9) +
```

```

0.25 * abs(COVER0.95 - 0.95) +
0.25 * abs(COVER0.99 - 0.99))

boxplots_sim_study(tmp_IAE, metric = "IAE_est",
  title = "Estimated IAE (Intervals)")

```



These results indicate the shortcoming of IAE: they are an excellent tool for assessing how well calibrated a method is, but they fail to take precision of the predictive interval into account. Thus, the baseline predictor will often be reasonably well calibrated when the marginal output of the computer model is even remotely bell-shaped.

References

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to algorithms*. MIT press, 2022.
- [2] D. Francom, B. Sansó, A. Kupresanin, and G. Johannesson, “Sensitivity analysis and emulation for functional data using bayesian adaptive splines,” *Statistica Sinica*, pp. 791–816, 2018.
- [3] E. Volkova, B. Iooss, and F. Van Dorpe, “Global sensitivity analysis for a numerical model of radionuclide migration from the RRC ‘kurchatov institute’ radwaste disposal site,” *Stochastic Environmental Research and Risk Assessment*, vol. 22, no. 1, pp. 17–31, 2008.
- [4] N. Klein, J. Colgan, E. Judge, K. Bhat, K. Myers, and E. Lawrence, “ATOMIC simulations and experimental data for basalt-like compounds.” Zenodo, Oct. 2020. doi: [10.25583/1670467](https://doi.org/10.25583/1670467).
- [5] K. N. Rumsey, Z. K. Hardy, C. Ahrens, and S. Vander Wiel, “Co-active subspace methods for the joint analysis of adjacent computer models,” *Technometrics*, vol. 67, no. 1, pp. 133–146, 2025.
- [6] D. L. Preston, D. L. Tonks, and D. C. Wallace, “Model of plastic deformation for extreme loading conditions,” *Journal of applied physics*, vol. 93, no. 1, pp. 211–220, 2003.

- [7] D. C. Francom *et al.*, “Simulation and emulation of x-ray diffraction from dynamic compression experiments,” *Journal of Dynamic Behavior of Materials*, vol. 7, no. LA–UR–19–31689, 2020.
- [8] G. R. Johnson, “A constitutive model and data for metals subjected to large strains, high strain rates and high temperatures,” in *Proceedings of the 7th international symposium on ballistics, am. Def. Prep. Org.(ADPA), netherlands, 1983*, 1983, pp. 541–547.
- [9] A. Hlobilová, S. Marelli, and B. Sudret, “Benchmark case datasets - two-dimensional heat diffusion model.” Zenodo, 2024. doi: [10.5281/zenodo.12701147](https://doi.org/10.5281/zenodo.12701147).
- [10] C. J. Smith *et al.*, “FAIR v1. 3: a simple emissions-based impulse response and carbon cycle model,” *Geoscientific Model Development*, vol. 11, no. 6, pp. 2273–2297, 2018.
- [11] K. N. Rumsey, D. Francom, and A. Shen, “Generalized bayesian MARS: Tools for stochastic computer model emulation,” *SIAM/ASA Journal on Uncertainty Quantification*, vol. 12, no. 2, pp. 646–666, 2024.
- [12] G. Hutchings, B. Sansó, J. Gattiker, D. Francom, and D. Pasqualini, “Comparing emulation methods for a high-resolution storm surge model,” *Environmetrics*, vol. 34, no. 3, p. e2796, 2023.
- [13] J. L. Brown and L. B. Hund, “Estimating material properties under extreme conditions by using bayesian model calibration with functional outputs,” *Journal of the Royal Statistical Society Series C: Applied Statistics*, vol. 67, no. 4, pp. 1023–1045, 2018.
- [14] M. Grosskopf *et al.*, “In-situ spatial inference on climate simulations with sparse gaussian processes,” in *ISAV’21: In situ infrastructures for enabling extreme-scale analysis and visualization*, 2021, pp. 31–36.
- [15] D. C. Francom, S. A. Vander Wiel, and B. P. Weaver, “Nuclear data dimension reduction,” Los Alamos National Lab.(LANL), Los Alamos, NM (United States), 2019.
- [16] D. Francom, B. Sansó, V. Bulaevskaya, D. Lucas, and M. Simpson, “Inferring atmospheric release characteristics in a large computer experiment using bayesian adaptive splines,” *Journal of the American Statistical Association*, vol. 114, no. 528, pp. 1450–1465, 2019.
- [17] M. J. Brown, A. Gowardhan, M. Nelson, M. Williams, and E. R. Pardyjak, “Evaluation of the QUIC wind and dispersion models using the joint urban 2003 field experiment dataset,” in *AMS 8th symp. Urban env, phoenix, USA*, 2009, pp. 10–16.
- [18] A. Fisher, C. Rudin, and F. Dominici, “All models are wrong, but many are useful: Learning a variable’s importance by studying an entire class of prediction models simultaneously,” *Journal of Machine Learning Research*, vol. 20, no. 177, pp. 1–81, 2019.
- [19] R. Xin, C. Zhong, Z. Chen, T. Takagi, M. Seltzer, and C. Rudin, “Exploring the whole rashomon set of sparse decision trees,” *Advances in neural information processing systems*, vol. 35, pp. 14071–14084, 2022.
- [20] C. Rudin *et al.*, “Amazing things come from having many good models,” *arXiv preprint arXiv:2407.04846*, 2024.
- [21] A. Visokay, E. Hock, K. Hoffman, and T. H. McCormick, “When you come to a fork in the road, take it: The rashomon effect for social scientists,” 2025.
- [22] L. Breiman, “Statistical modeling: The two cultures (with comments and a rejoinder by the author),” *Statistical science*, vol. 16, no. 3, pp. 199–231, 2001.
- [23] E. D. Dolan and J. J. Moré, “Benchmarking optimization software with performance profiles,” *Mathematical programming*, vol. 91, no. 2, pp. 201–213, 2002.
- [24] J. J. Moré and S. M. Wild, “Benchmarking derivative-free optimization algorithms,” *SIAM Journal on Optimization*, vol. 20, no. 1, pp. 172–191, 2009.

- [25] A. Marrel and B. Iooss, “Probabilistic surrogate modeling by gaussian process: A review on recent insights in estimation and validation,” *Reliability Engineering & System Safety*, vol. 247, p. 110094, 2024.